



Lecture 9: Image Generation Models (Part 1)

Email: idanschwartz@gmail.com or
idan.schwartz@biu.ac.il

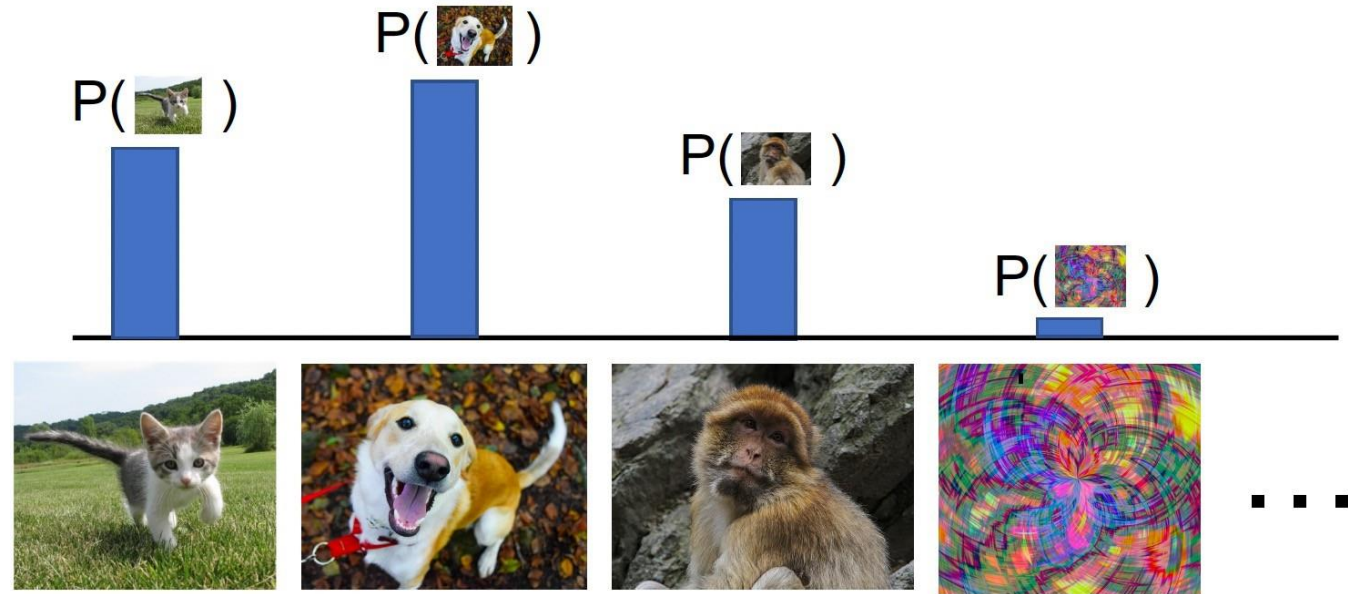
Recall: Discriminative vs Generative models

Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

Learn a probability distribution $p(x)$



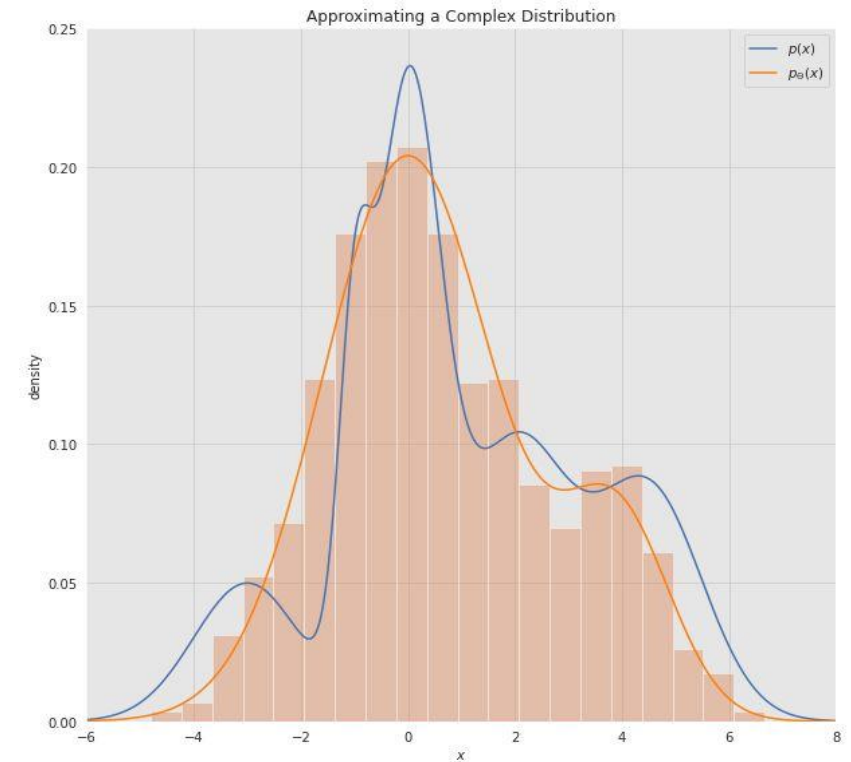
- Generative model: All possible images compete with each other for probability mass
- Model can “reject” unreasonable inputs by assigning them small values

Recall: Generative Modeling

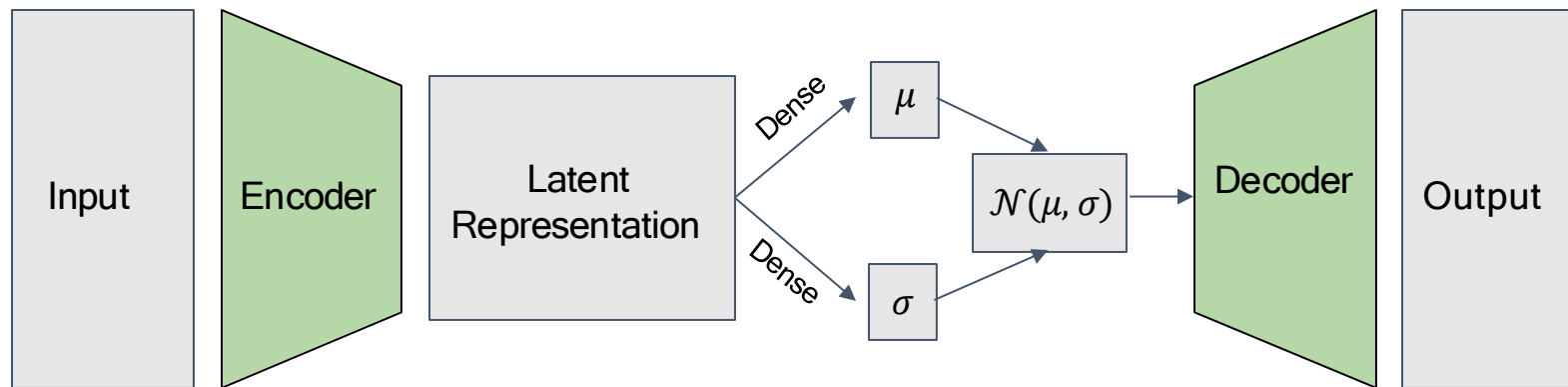
- The goal of generative modeling - learning a model of a distribution from which we can generate new samples.
- Given $x \sim p(x)$ we want to learn $p_\theta(x) \approx p(x)$ (modeling)
- Then, we can generate new samples $x \sim p_\theta(x)$ (generation)

Generative Modeling with Gaussians

- The distribution $p(x)$ is **complex**
- We want to work with **simple** distributions (such as Gaussians)
- **Idea:** learn a NN to transform a simple distribution sample into a complex one



Variational Auto-encoder



Weighted Combination of Losses

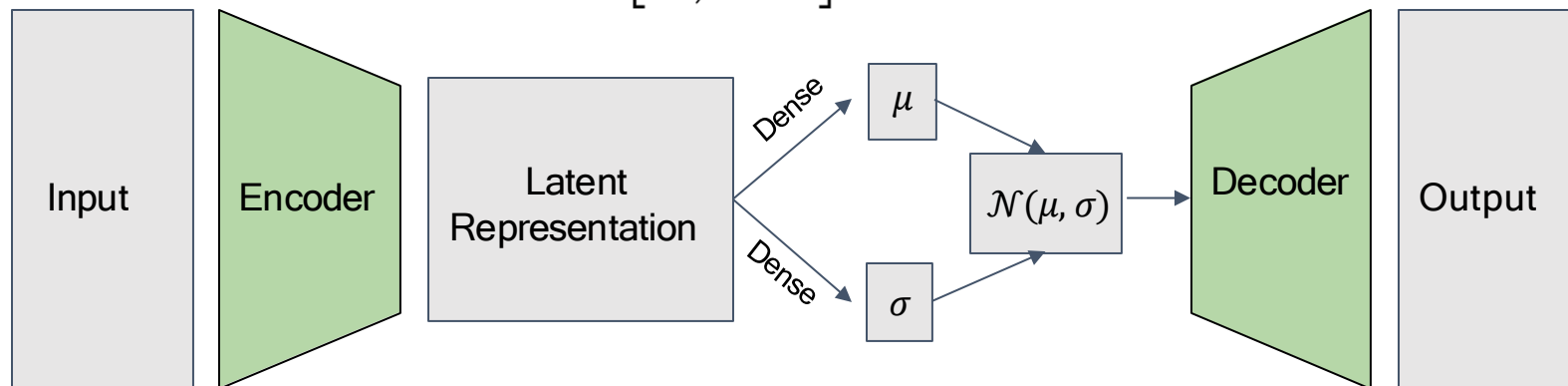
L_1 = loss associated with producing output similar to input

L_2 = loss associated with producing output with some variation to input

$$L = L_1 + \lambda L_2$$

Total Loss:

↑
 $\lambda \in [0, \infty]$



VAE Losses, Defined

We have seen L_1 before: this is just the autoencoder reconstruction loss

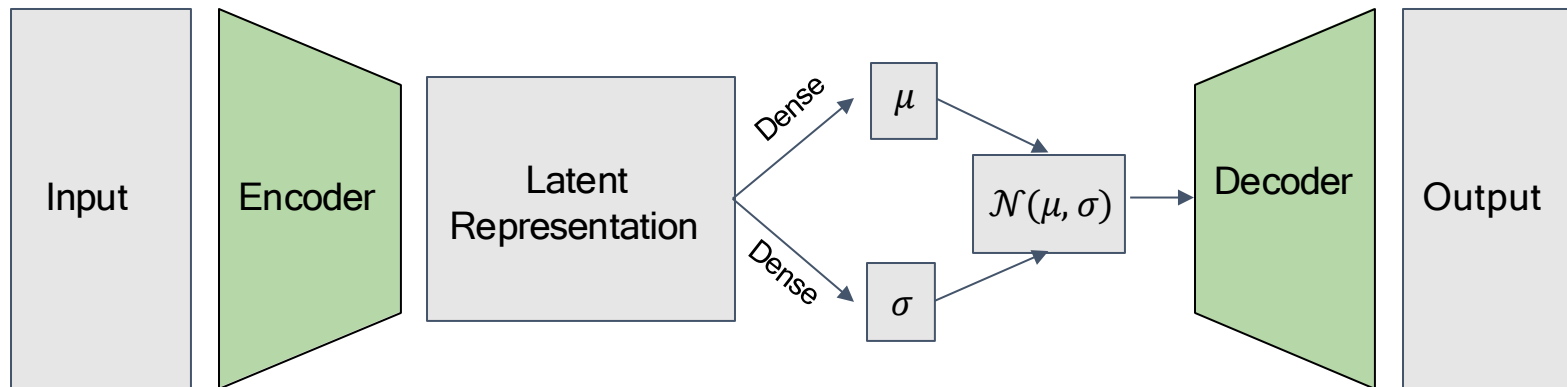
$$L_1(x, \hat{x}) = ||x - \hat{x}||_2^2$$

But with L_2 , it's not so clear. How do we measure how much variation our output would have?

$$L_2(??, ??) = ??$$

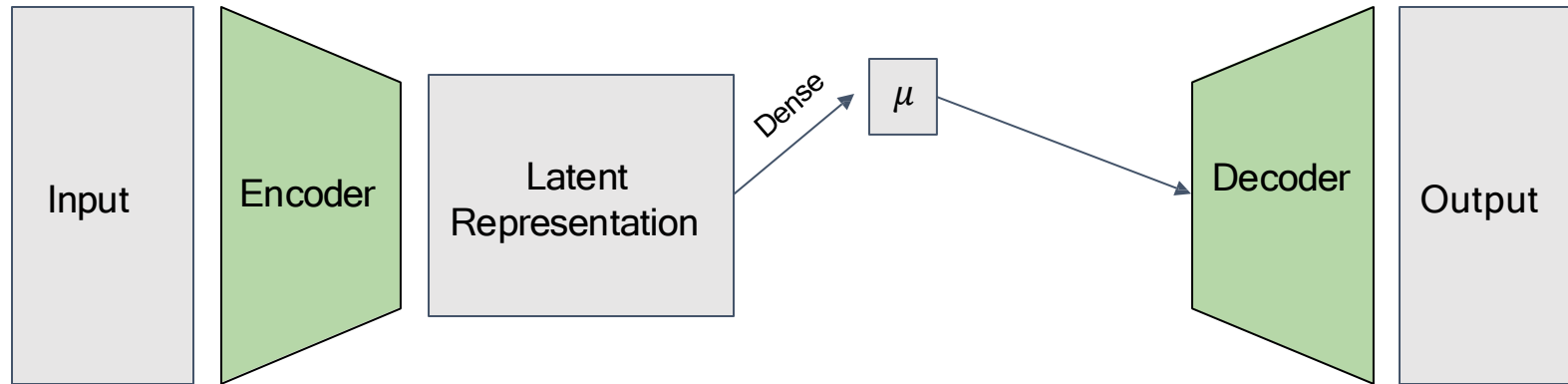
Defining the L_2 Loss...

- To get variation, we need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions



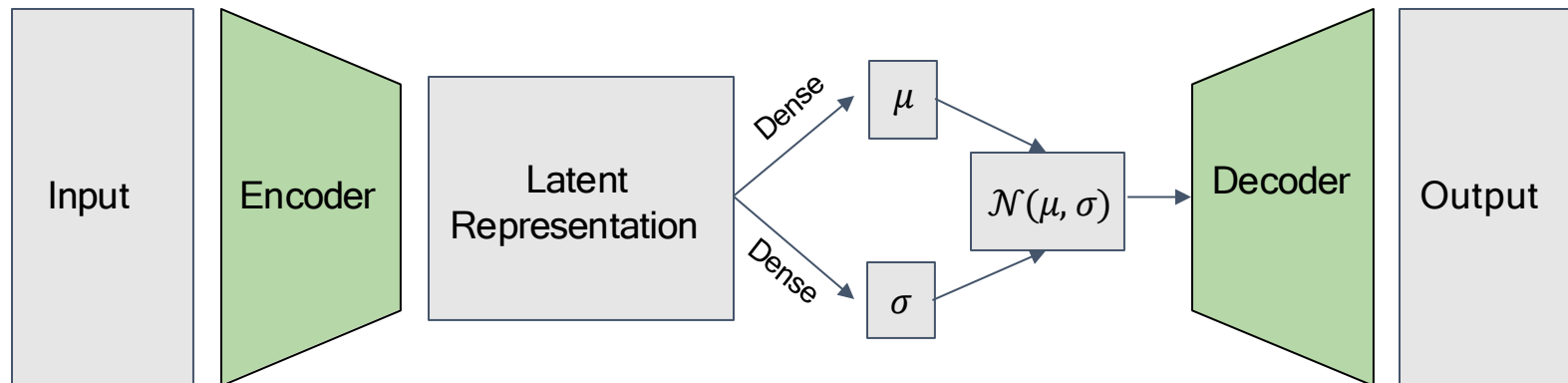
Defining the L_2 Loss...

- To get variation, we need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions
 - Behaves the same as a regular autoencoder!



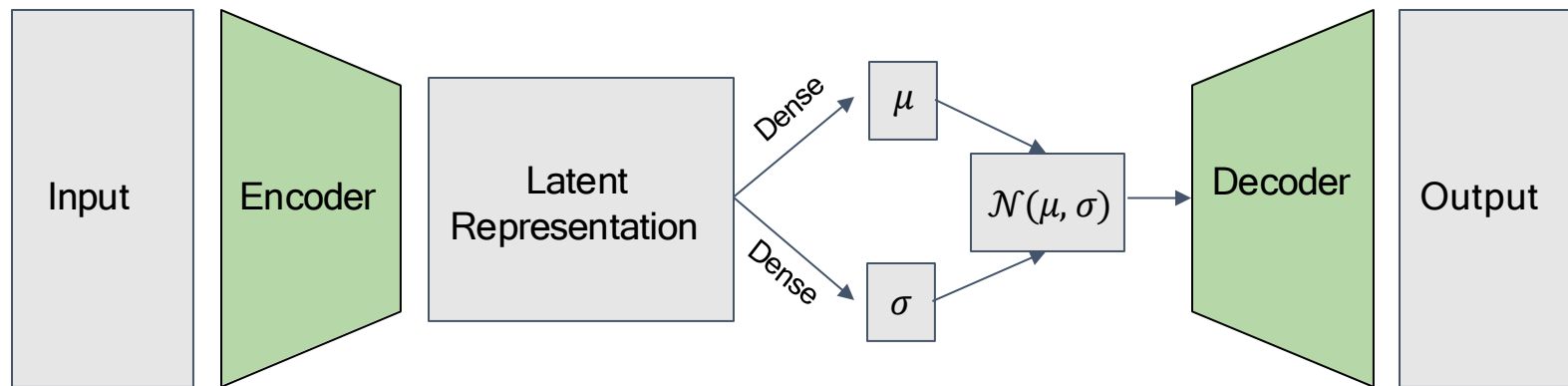
Defining the L_2 Loss...

- To get variation, we need a loss that encourages $\sigma > 0$
 - If we don't do this, L_1 will drive σ to zero in an effort to produce the best reconstructions
- But how big should we encourage σ to be?
- And for that matter, what we do about μ ?



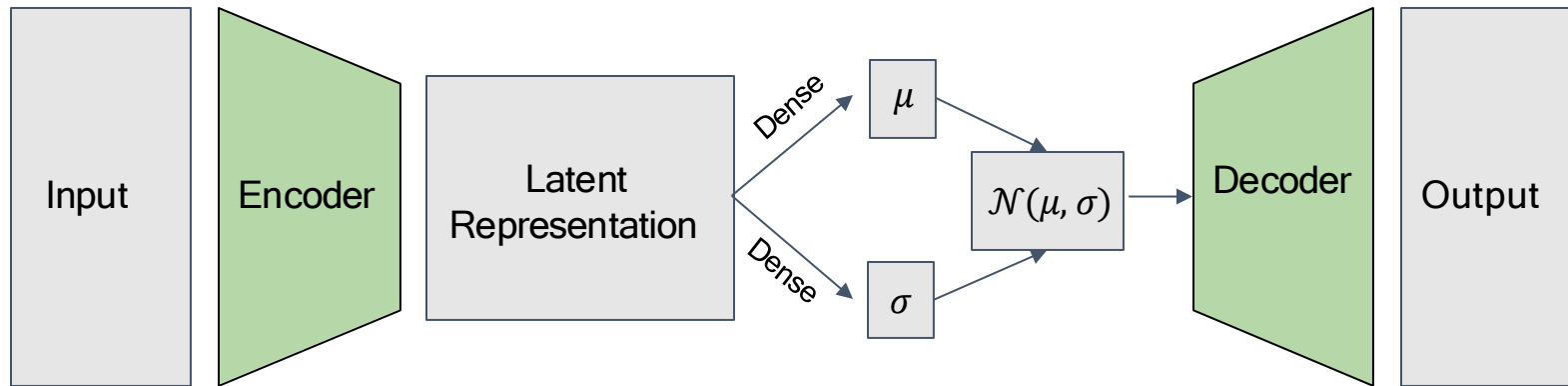
Defining the L_2 Loss...

- **The idea:** make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$
 - we'll see later that having some light pressure to make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$ will have some beneficial properties



Defining the L_2 Loss...

- **how** do we make $\mathcal{N}(\mu, \sigma)$ close to $\mathcal{N}(0, 1)$?
- More generally: how do we measure the difference between two probability distributions?



Kullback–Leibler (KL) Divergence

Measures the difference between any two probability distributions

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

What this says:

- “Everywhere that p has probability density...”
- “...the difference between p and q should be small”
 - Difference in log probabilities (remember that $\log \left(\frac{a}{b} \right) = \log(a) - \log(b)$)

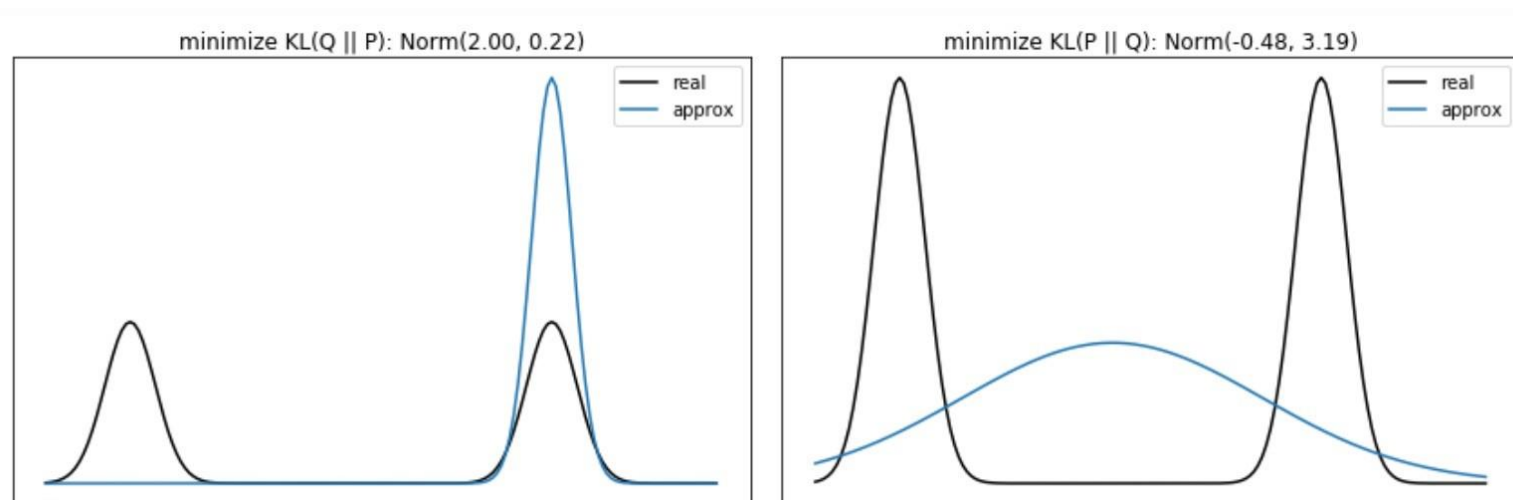
More on KL Divergence: <https://jessicastringham.net/2018/12/27/KL-Divergence/>

Kullback–Leibler (KL) Divergence

Measures the difference between any two probability distributions

$$D_{KL}(P||Q) = \int_{-\infty}^{\infty} p(x) \log \left(\frac{p(x)}{q(x)} \right) dx$$

- Note that this is not symmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$



Kullback–Leibler (KL) Divergence

- Expensive to compute, in general (no closed form, have to numerically approximate the integral)
- But! There is a closed form for Gaussians:

$$D_{KL}(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

k is the dimensionality of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ (e.g. $k = 100$ when $\mu \in \mathbb{R}^{100}$)

We won't derive the equation above, but let's convince ourselves it behaves how we expect it to behave

Kullback–Leibler (KL) Divergence

- Expensive to compute, in general (no closed form, have to numerically approximate the integral)
- But! There is a closed form for Gaussians:

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Derive the expression for (1) $\sigma = 1$ and (2) $\mu=0$

KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\sigma = 1$

$$\begin{aligned} D_{KL}(\mathcal{N}(\mu, 1) || \mathcal{N}(0, 1)) &= \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + 1^2 - \ln(1) - 1) \\ &= \frac{1}{2} \sum_{i=1}^k \mu_i^2 \end{aligned}$$

The expression is minimized $\mu = 0$ (which is what we want)

KL Divergence for Two Gaussians

$$D_{KL}(\mathcal{N}(\mu, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\mu_i^2 + \sigma_i^2 - \ln \sigma_i^2 - 1)$$

Let's take the case $\mu = 0$

$$D_{KL}(\mathcal{N}(0, \sigma^2) || \mathcal{N}(0, 1)) = \frac{1}{2} \sum_{i=1}^k (\sigma_i^2 - \ln(\sigma_i^2) - 1)$$

This expression is minimized when $\sigma = 1$ (which is also what we want)

The Final VAE Loss Function

We now have all the tools necessary to construct our loss function.

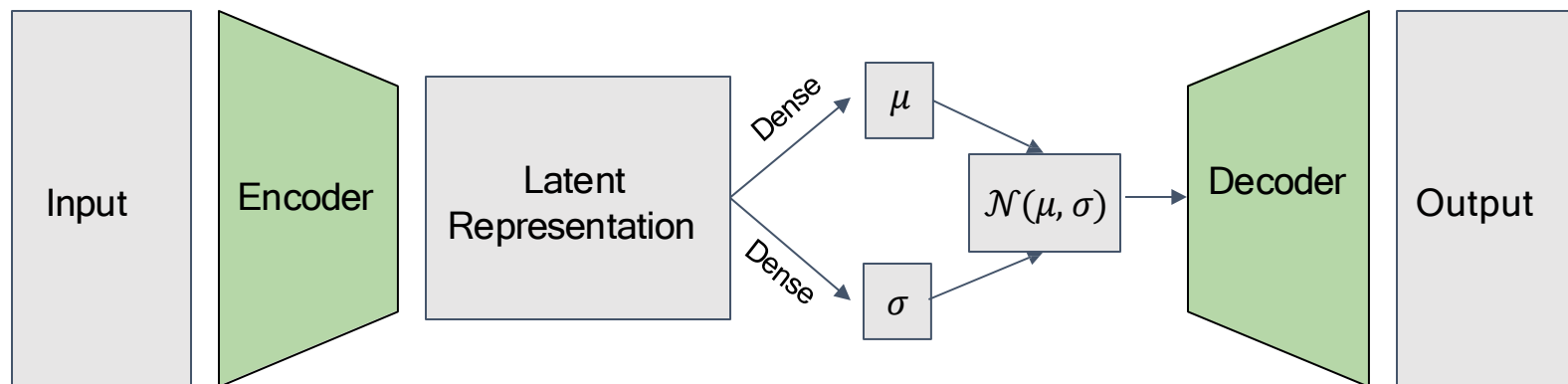
$$L = L_1 + \lambda L_2 \quad \lambda \in [0, \infty]$$

Which turns into this:

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$

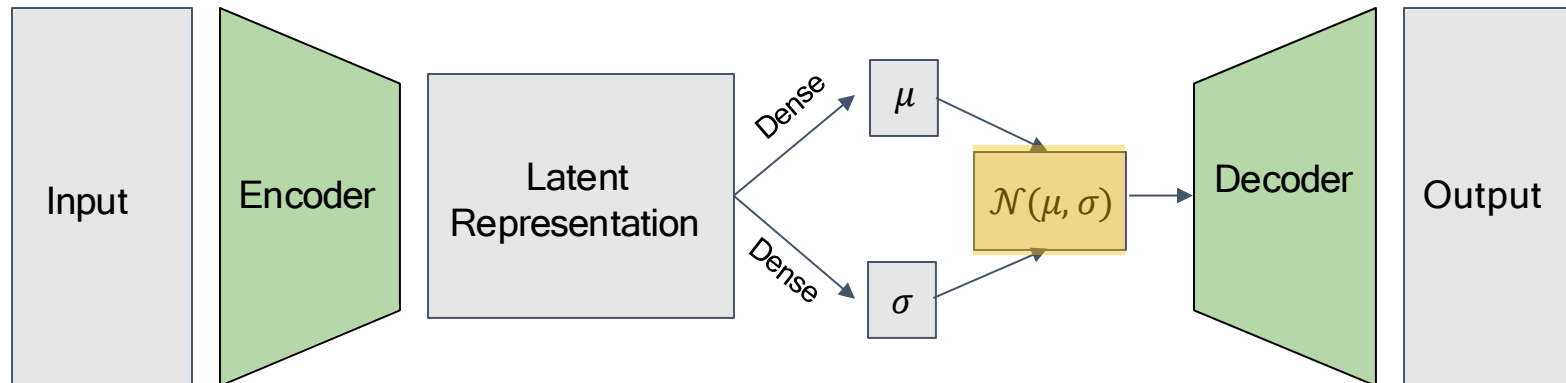
Putting it all together

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$



Ah, but there's a catch:

- To update the weights of the encoder, we have to backprop through a random sampling operation
- Sampling a random value seems not differentiable...



The Reparameterization Trick

A nice property of Gaussian distributions: if we sample $z \sim \mathcal{N}(\mu, \sigma)$ we can rewrite it as:

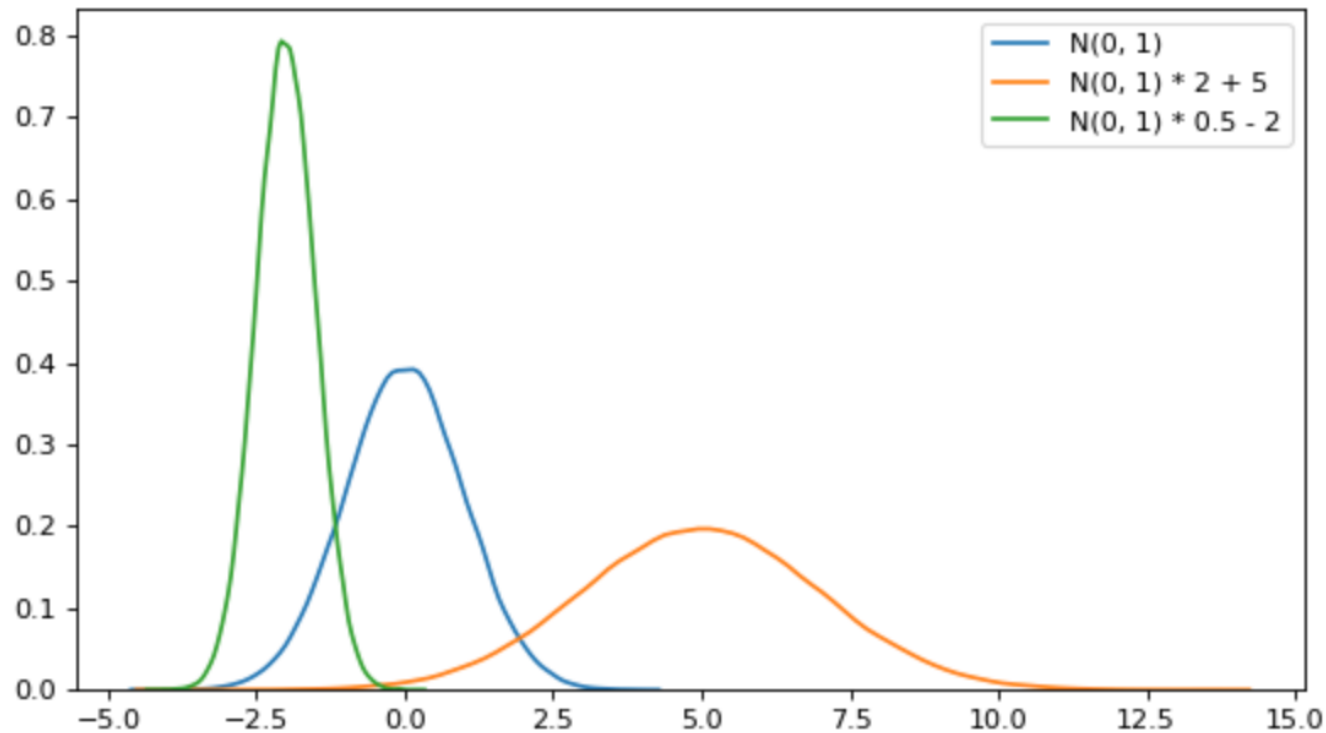
$$z = \mu + \epsilon \cdot \sigma$$

Where

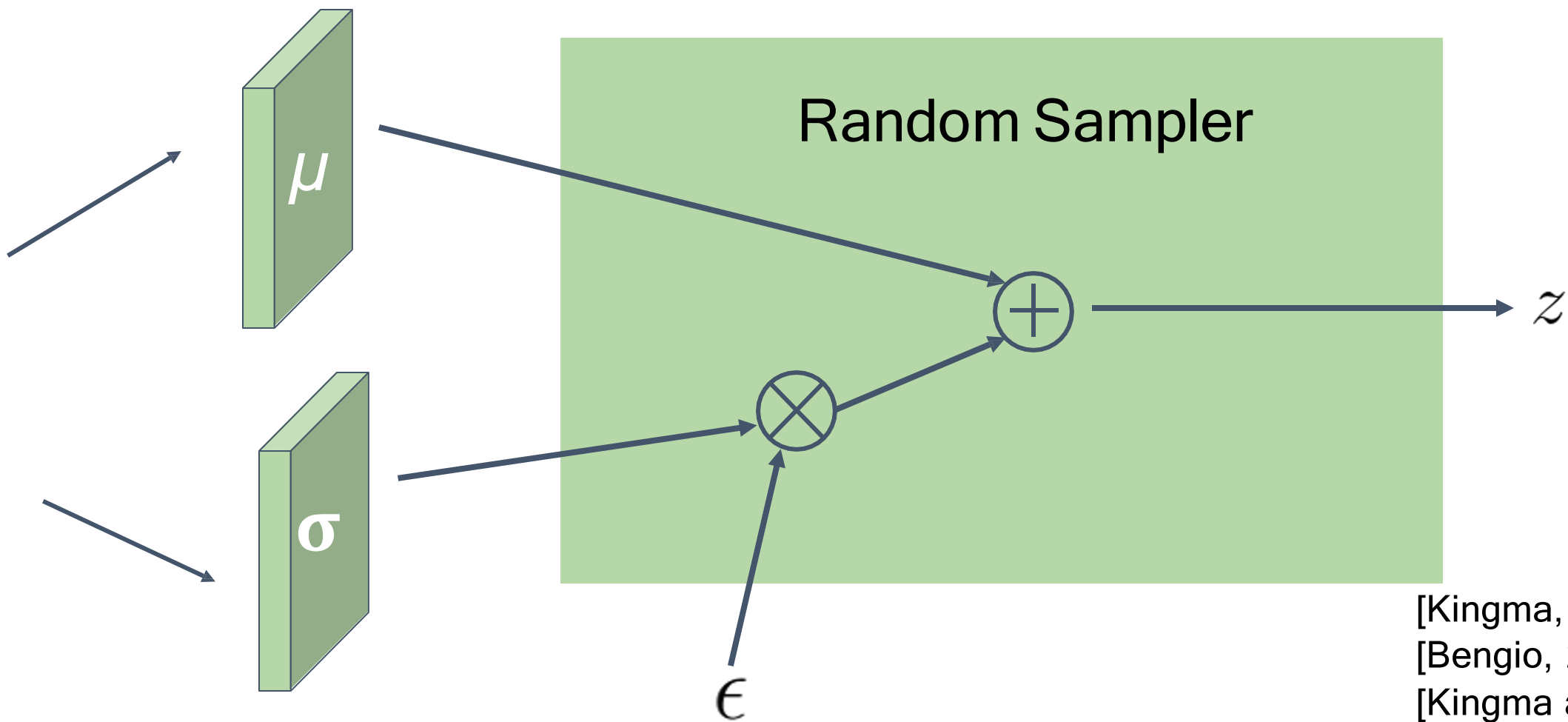
$$\epsilon \sim \mathcal{N}(0, 1)$$

Reparameterization Trick

Re-write $x \sim \mathcal{N}(x|\mu, \sigma^2)$ as $x = \mu + \sigma\epsilon$ where $\epsilon \sim \mathcal{N}(x|0, I)$

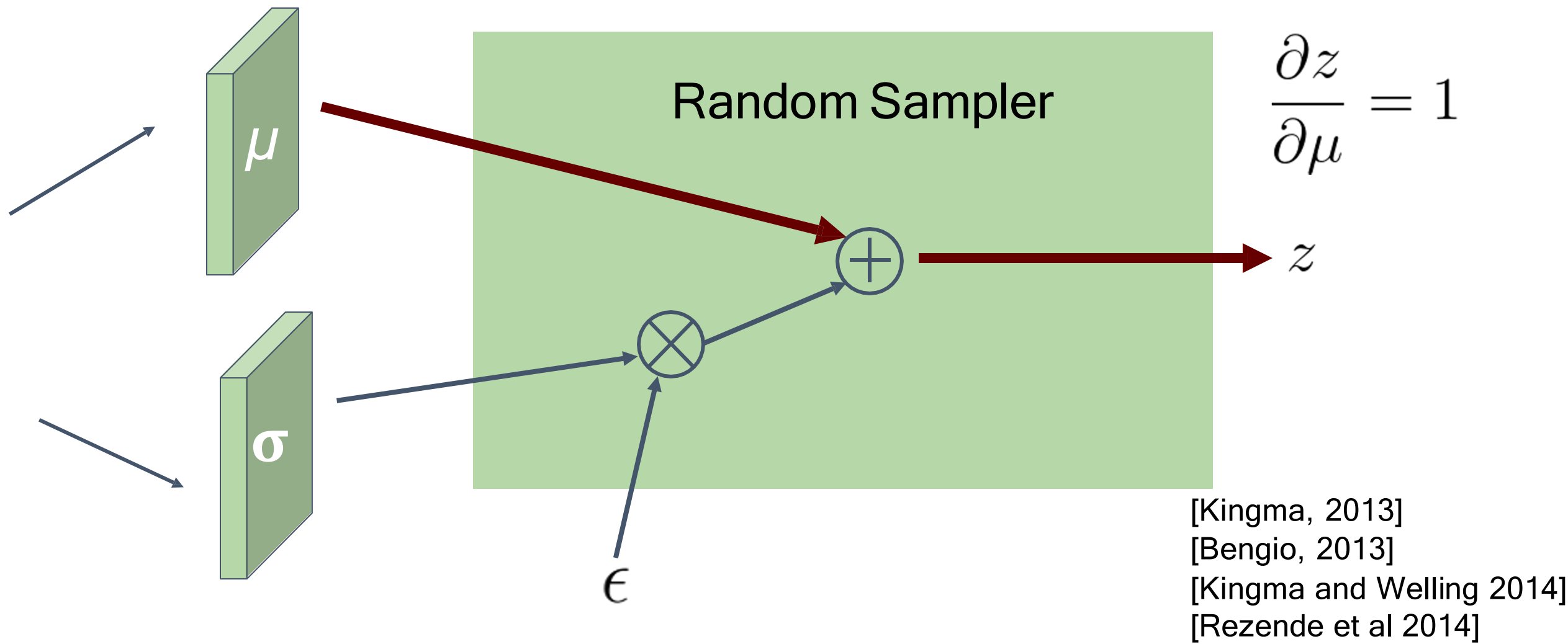


Random Sampler with Reparameterization Trick

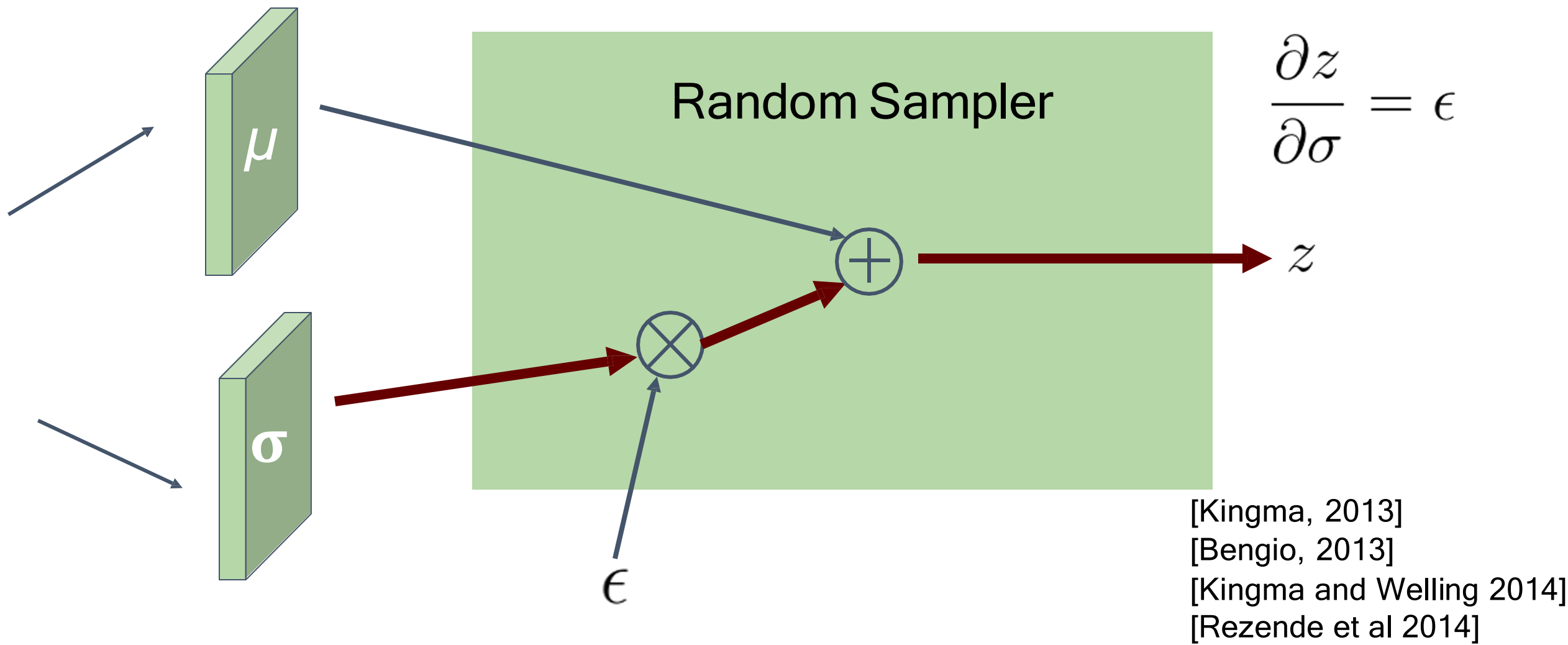


[Kingma, 2013]
[Bengio, 2013]
[Kingma and Welling 2014]
[Rezende et al 2014]

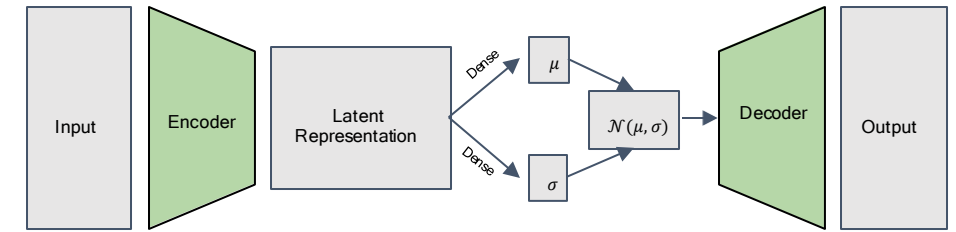
Random Sampler with Reparameterization Trick



Random Sampler with Reparameterization Trick



One more practical detail



Let's again consider our sampling operation

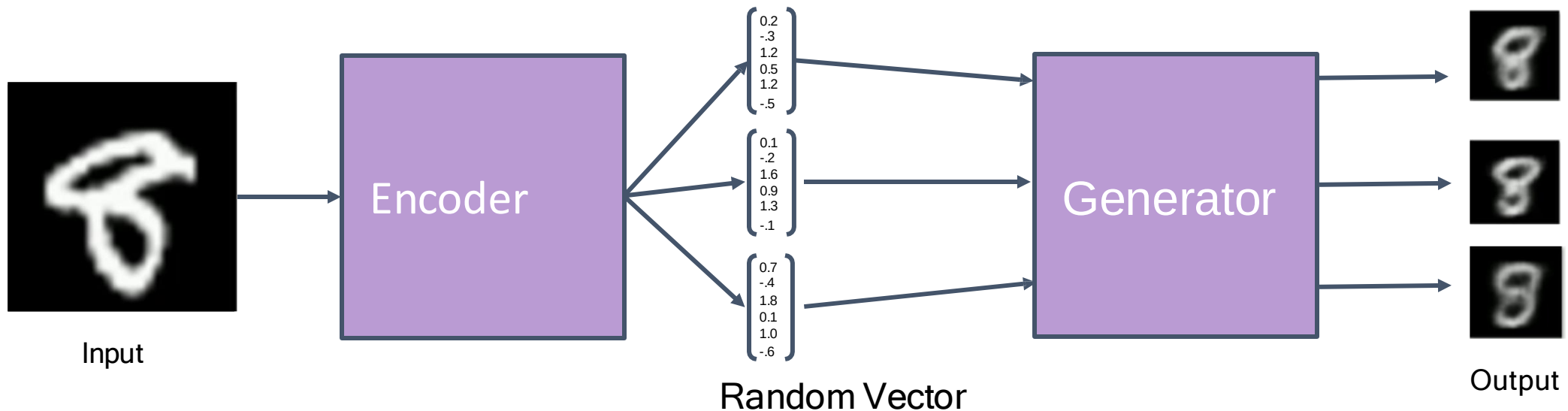
$$z \sim \mathcal{N}(\mu, \sigma)$$

$\mu_i \in [-\infty, \infty]$ $\sigma_i \in [0, \infty]$

- Nothing prevents the neural network from outputting ***negative*** values for the standard deviation.
- Instead of predicting σ , we will instead predict $\log(\sigma^2)$. This ensures that every $\sigma_i \in [0, \infty]$
 - i.e. just treat the output of the Dense layer as if it is $\log(\sigma^2)$

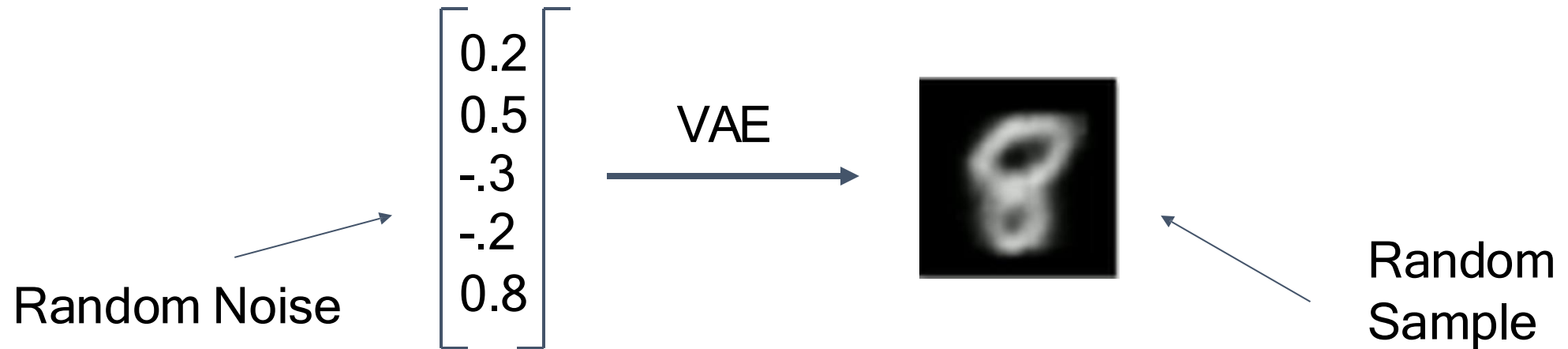
Sampling from a VAE

- We can use a trained VAE to generate random variants of an input data point



Sampling from a VAE

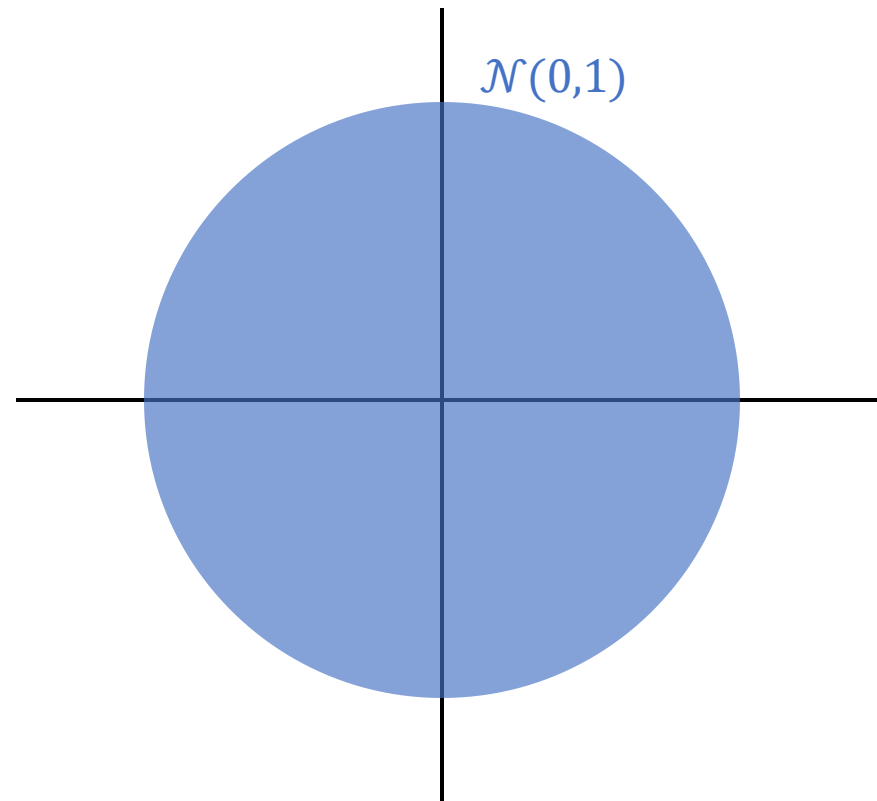
But ultimately, we want to draw random samples from a VAE



How can we do this?

This is where our particular choice of training loss will pay off

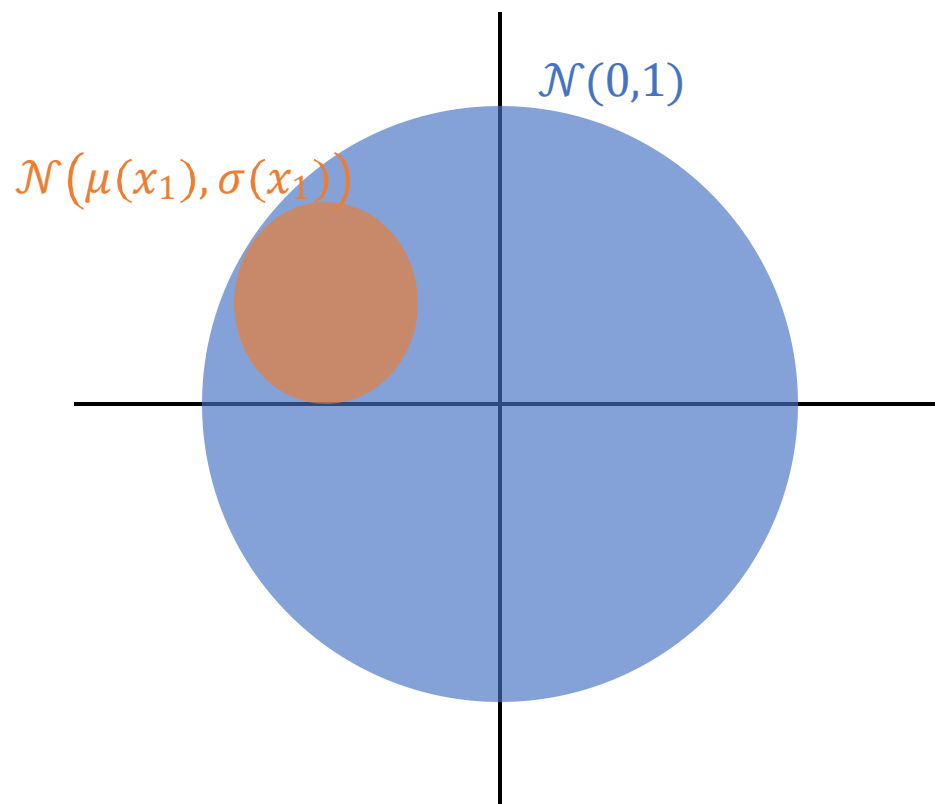
Encoding different points into latent space



Let this circle represent the probability density of a unit Gaussian in latent space

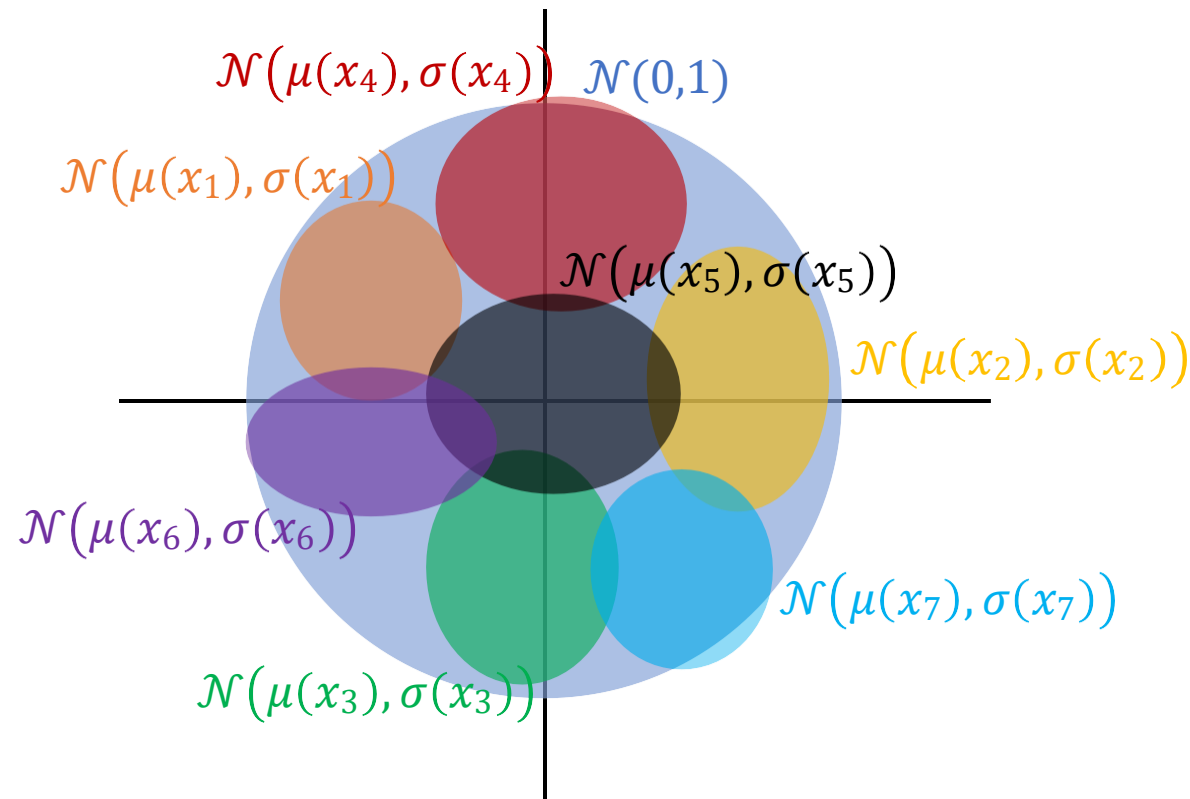
Encoding different points into latent space

Let this circle represent the probability density of the $\mathcal{N}(\mu, \sigma)$ distribution that the encoder predicts given an input data point x_1



Encoding different points into latent space

$$L = ||x - \hat{x}||_2^2 + \lambda D_{KL}(\mathcal{N}(\mu, \sigma), \mathcal{N}(0, 1))$$

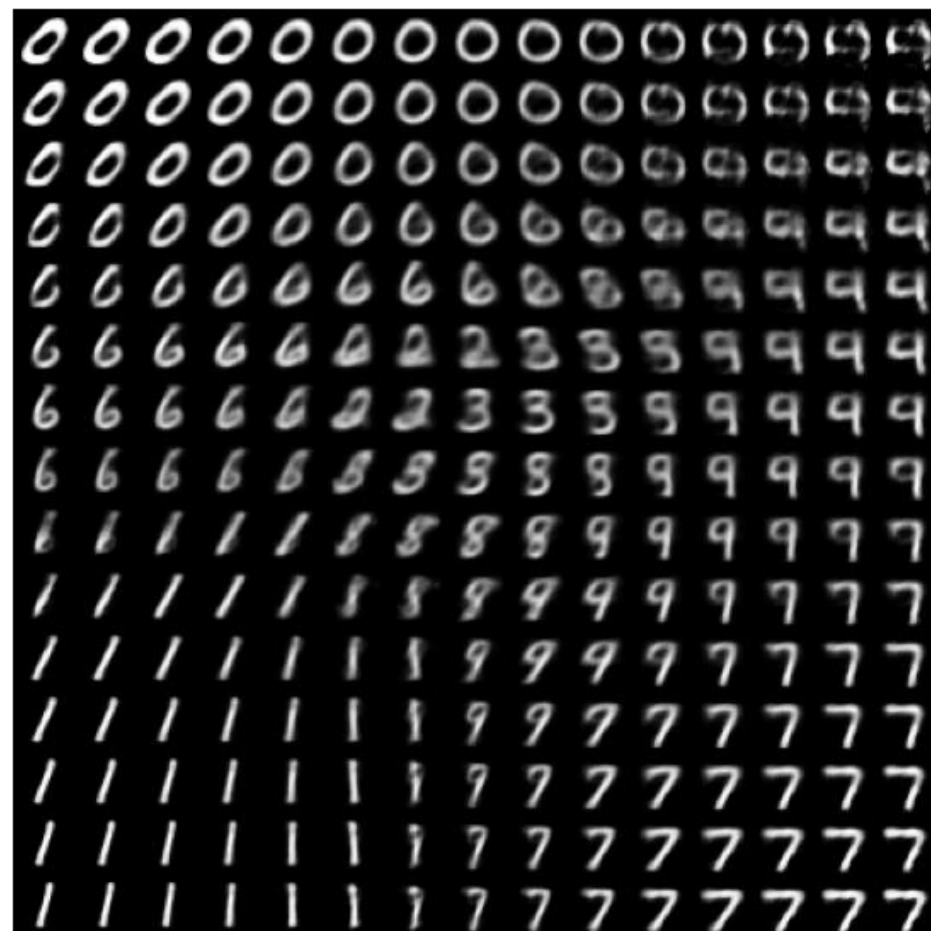


Because of our KL divergence loss, the $\mathcal{N}(\mu, \sigma)$ for any input data point has to be somewhat similar to $\mathcal{N}(0,1)$

So, if we sample a point from $\mathcal{N}(0,1)$, it is very likely to fall within one of these encoded distributions from the training set...

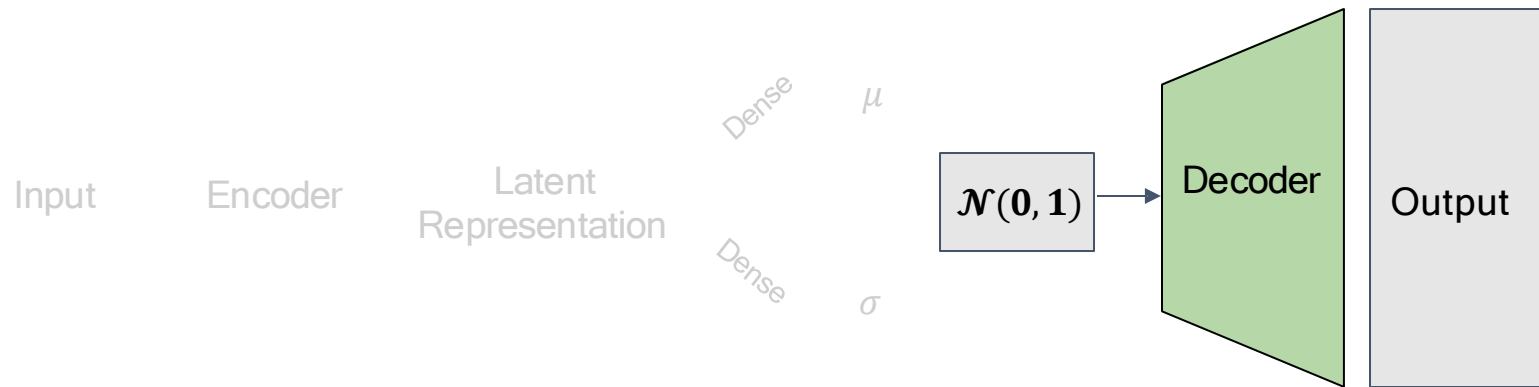
...which the decoder has been trained to reconstruct well!

VAE on MNIST



(trained by Alec Radford 2015)

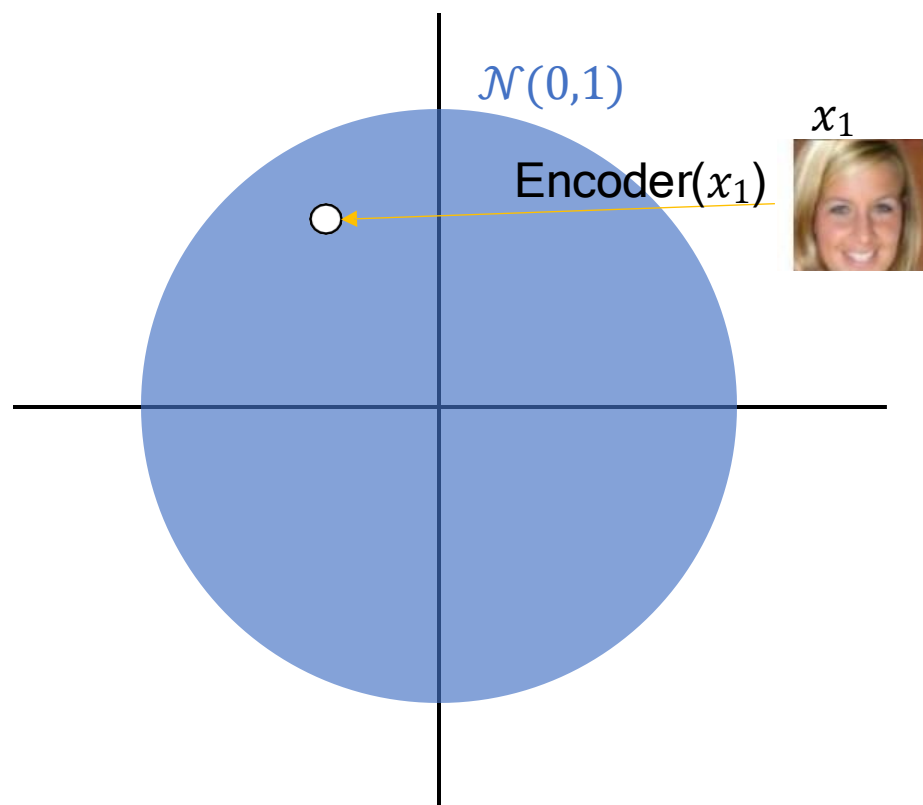
Sampling from a VAE



- Discard this part of the network...
- ...and set $(\mu, \sigma) = (0, 1)$

Latent Space Interpolation

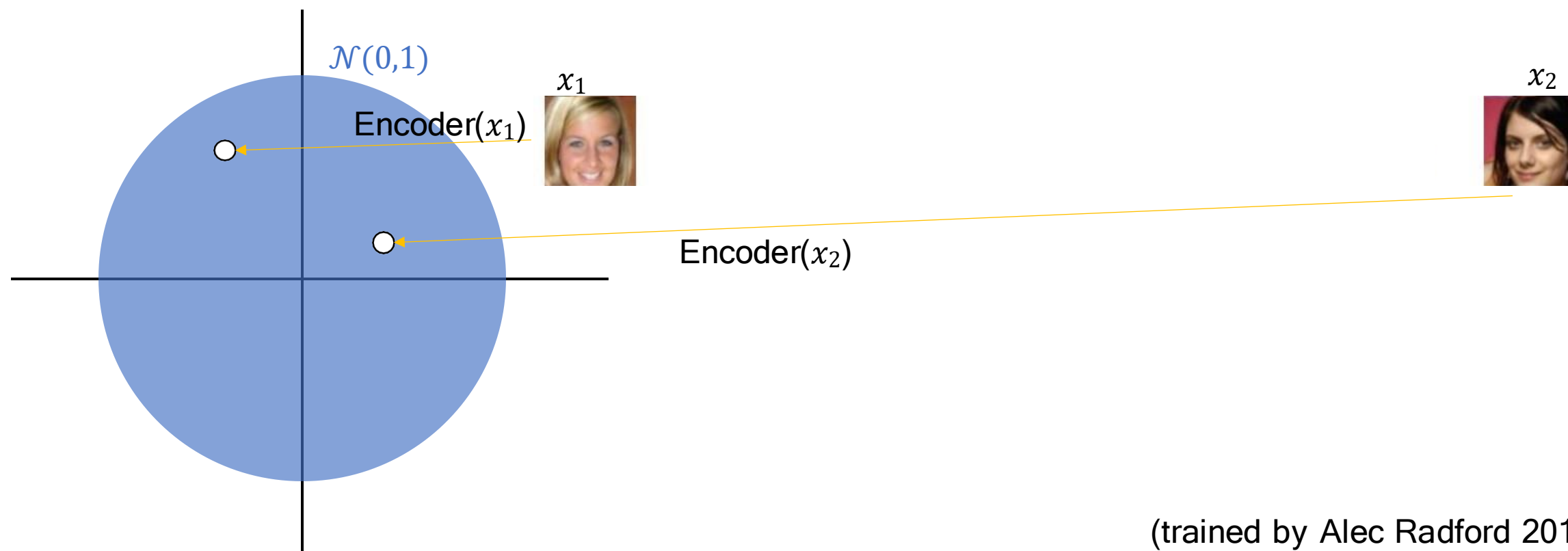
- Trace a linear path between two points in latent space, put all points along the path into the decoder



(trained by Alec Radford 2015)

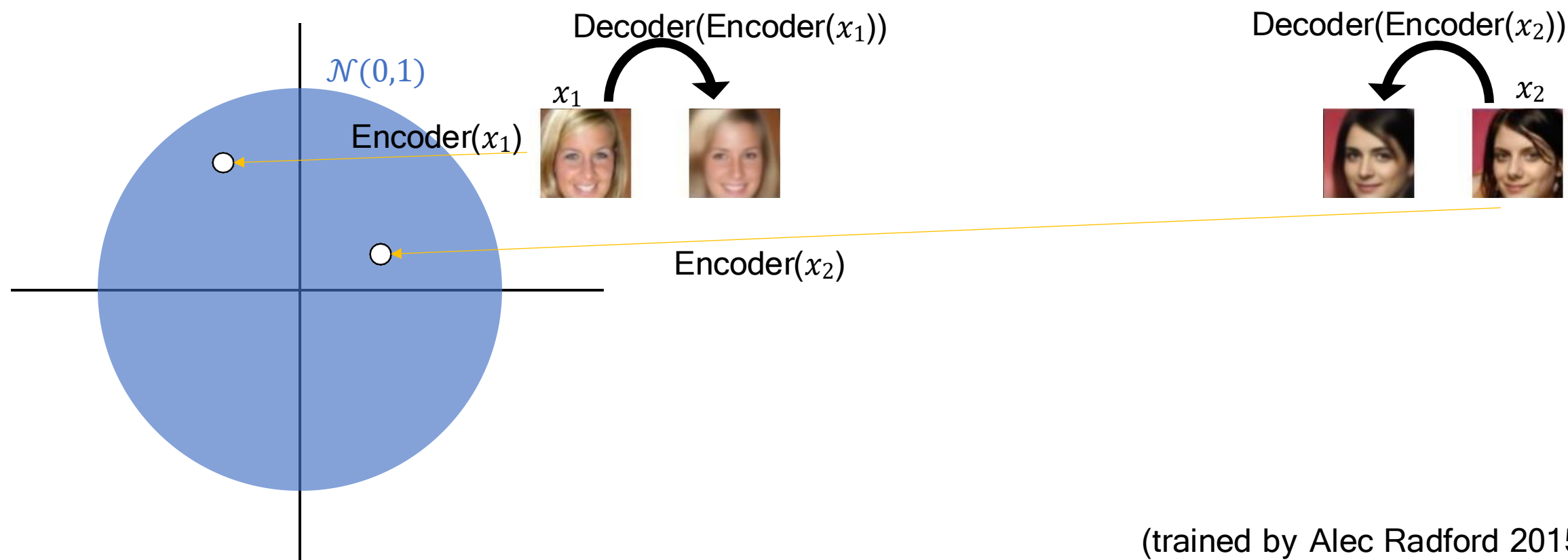
Latent Space Interpolation

- Trace a linear path between two points in latent space, put all points along the path into the decoder



Latent Space Interpolation

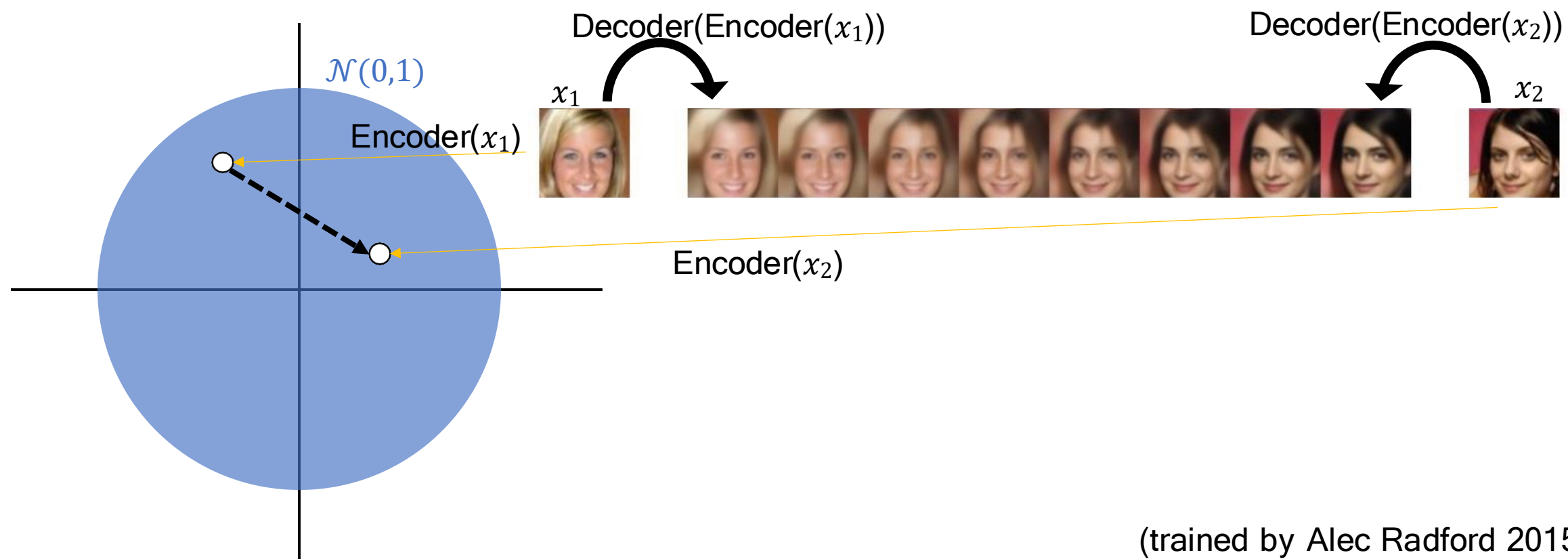
- Trace a linear path between two points in latent space, put all points along the path into the decoder



(trained by Alec Radford 2015)

Latent Space Interpolation

- Trace a linear path between two points in latent space, put all points along the path into the decoder

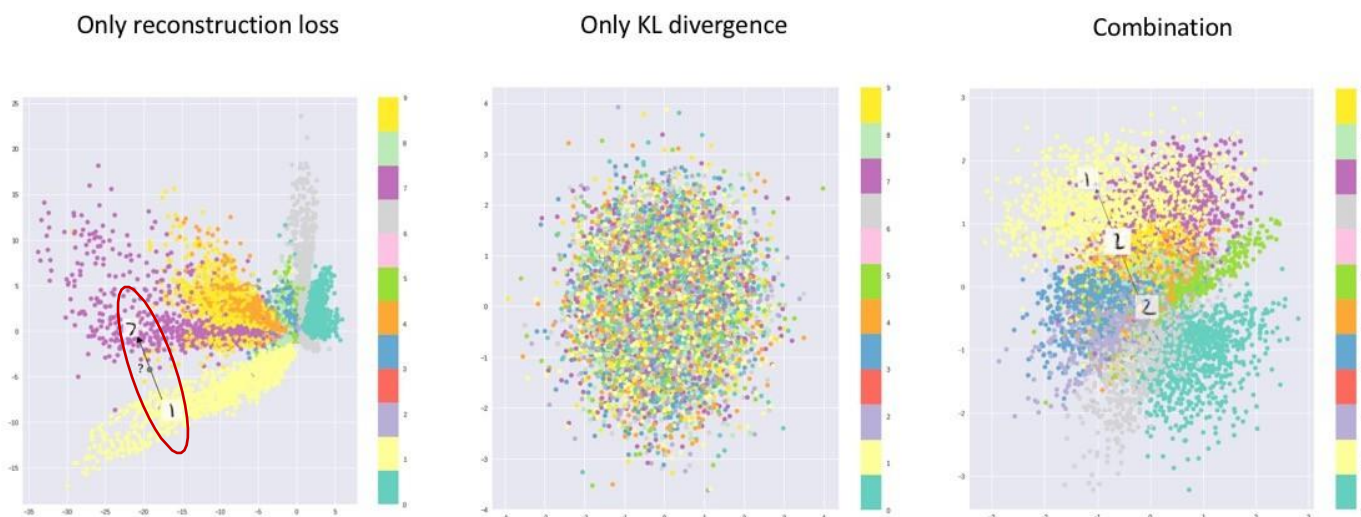


(trained by Alec Radford 2015)

Latent Space Interpolation

- Can also try it with a regular autoencoder
 - Doesn't work as well
 - ***Why not?***
 - The KL divergence loss regularizes the shape of the latent space. Without it, a regular autoencoder might have “empty” pockets of latent space

Linear interpolation has to cross a pocket of empty space (where the behavior of the decoder is not well defined)



Discriminative vs Generative Models

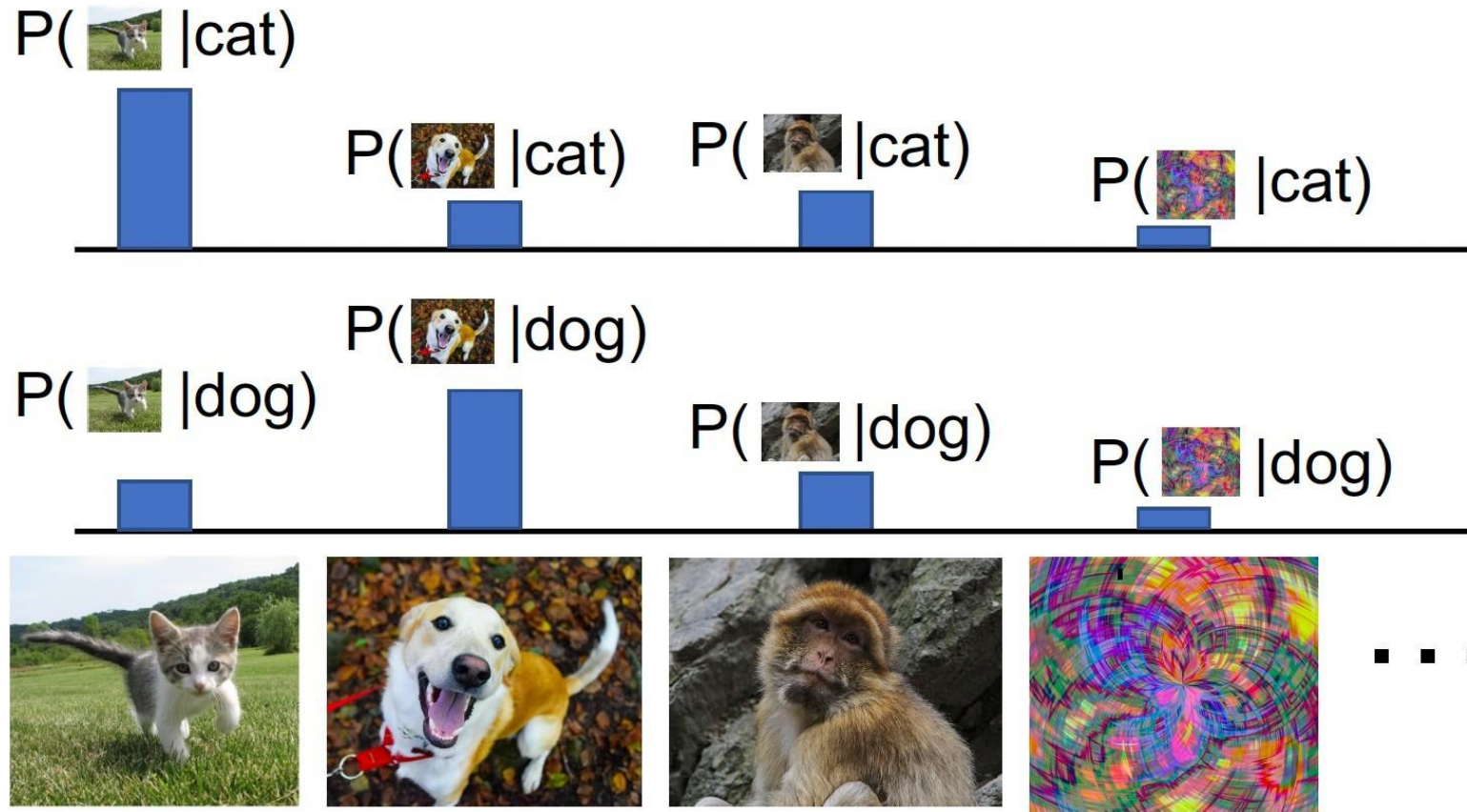
Discriminative Model:

Learn a probability distribution $p(y|x)$

Generative Model:

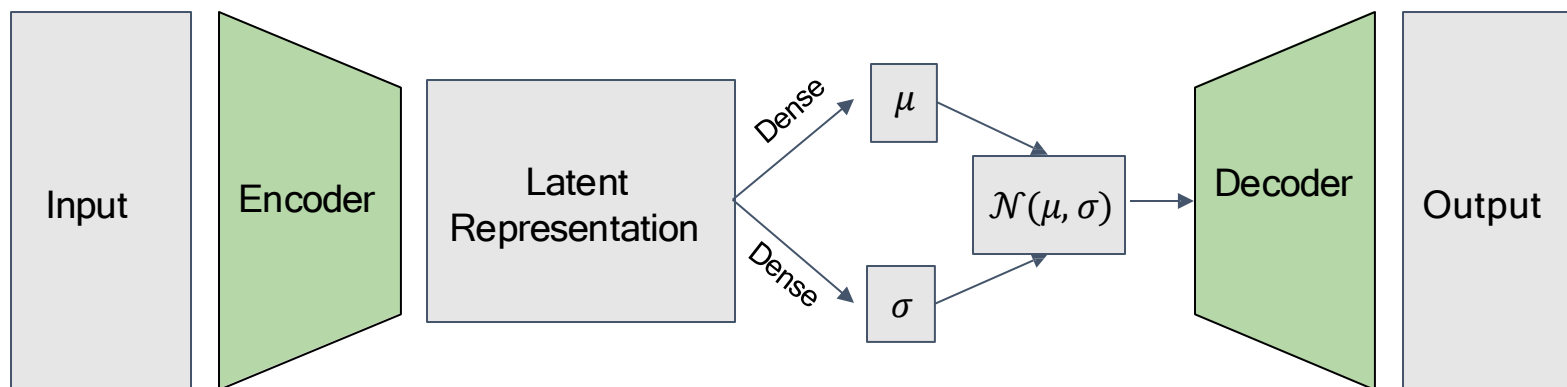
Learn a probability distribution $p(x)$

Conditional Generative Model: Learn $p(x|y)$

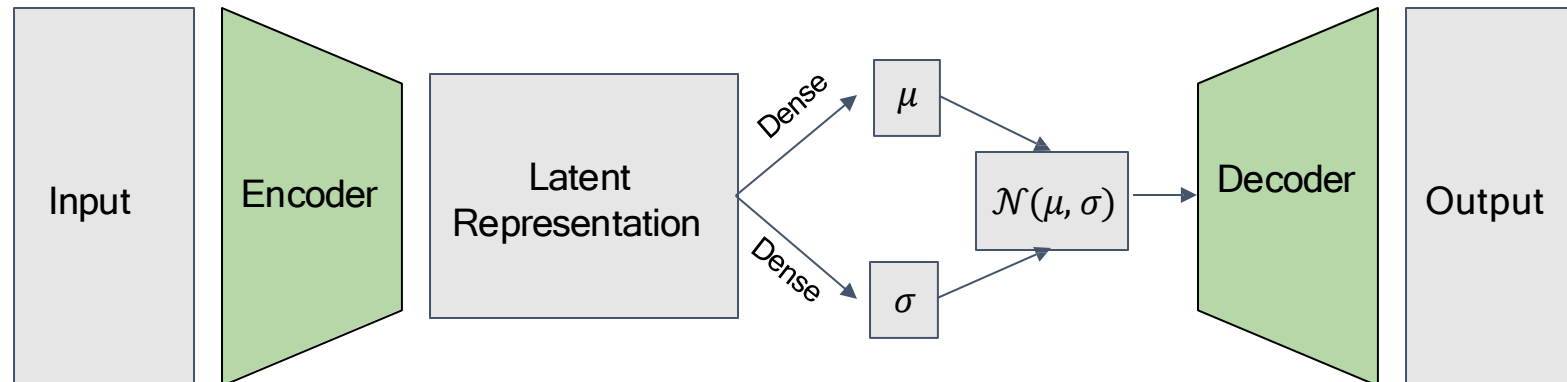
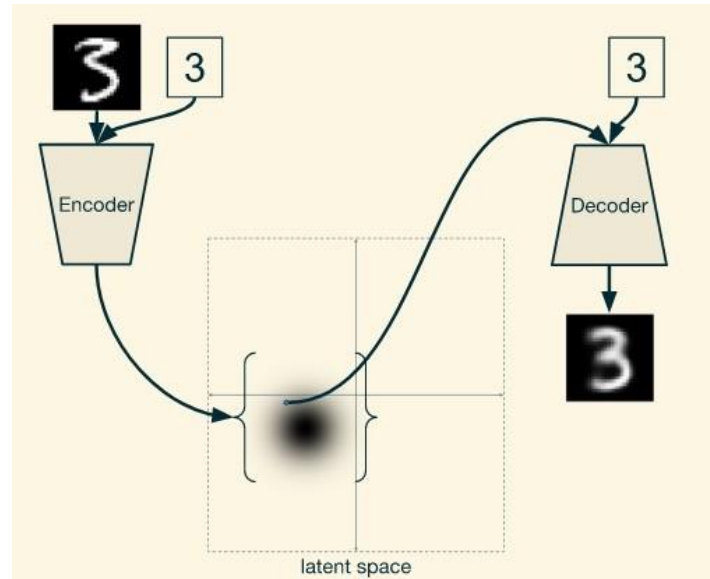


Conditional Generative Model: Each possible label induces a competition among all images

Conditional VAE

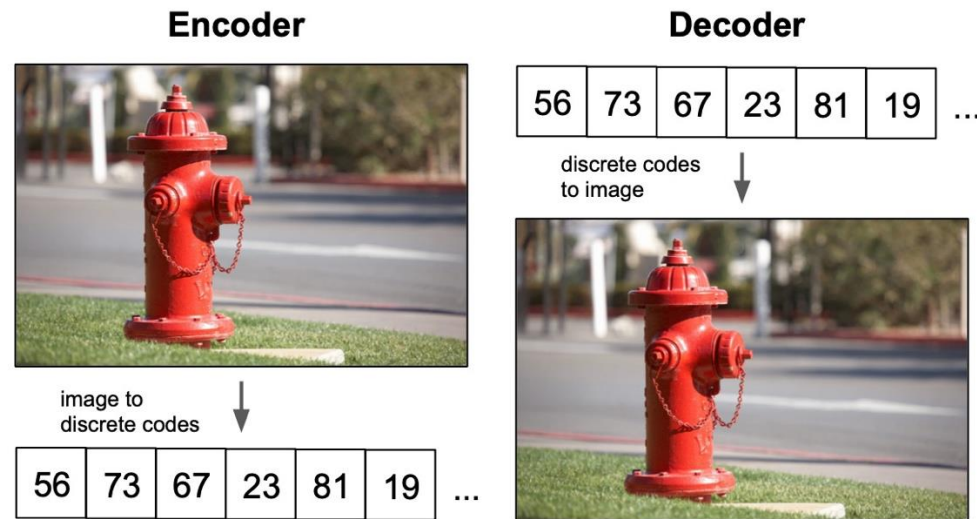


Conditional VAE

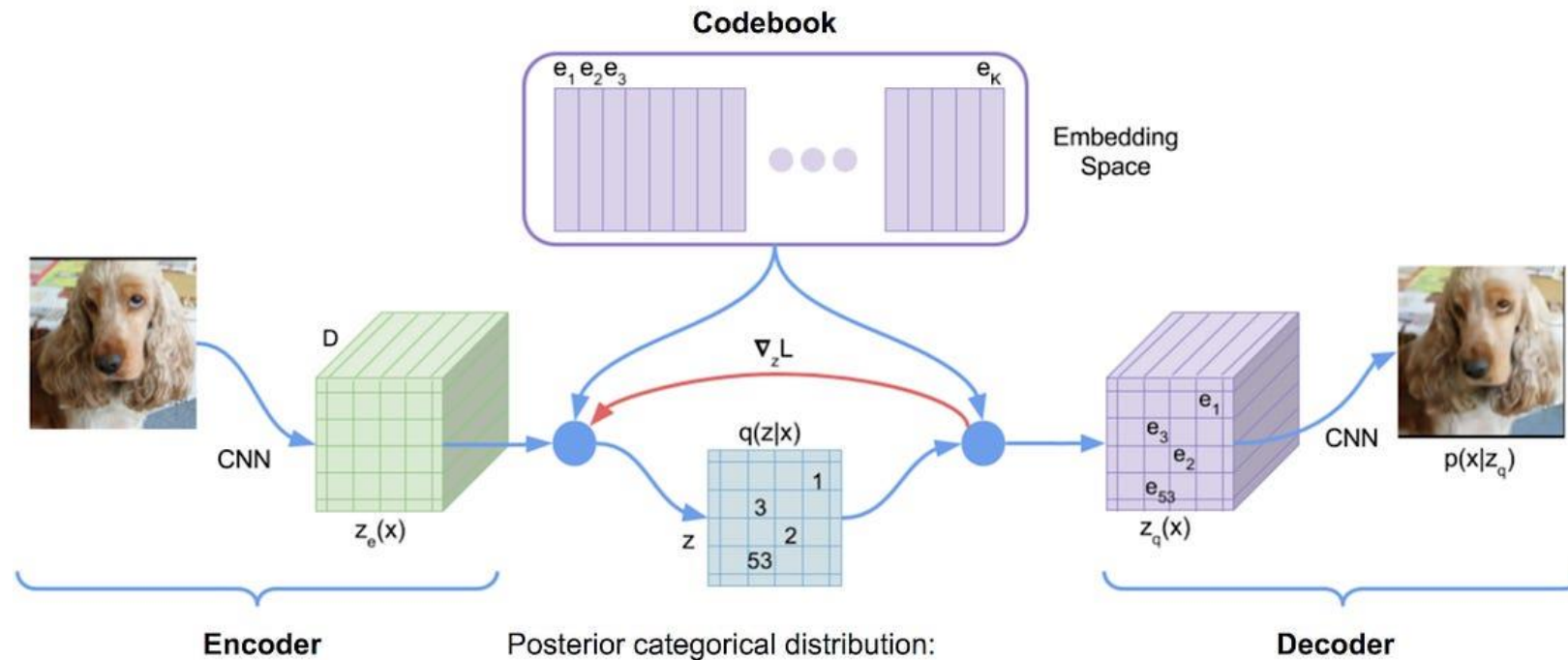


Vector Quantized Variational Autoencoder (VQ-VAE)

- So far, we have seen how continuous vector spaces can be used to represent the latents in an autoencoder.
- But latents do not necessarily need to be continuous vectors



VQ-VAE training

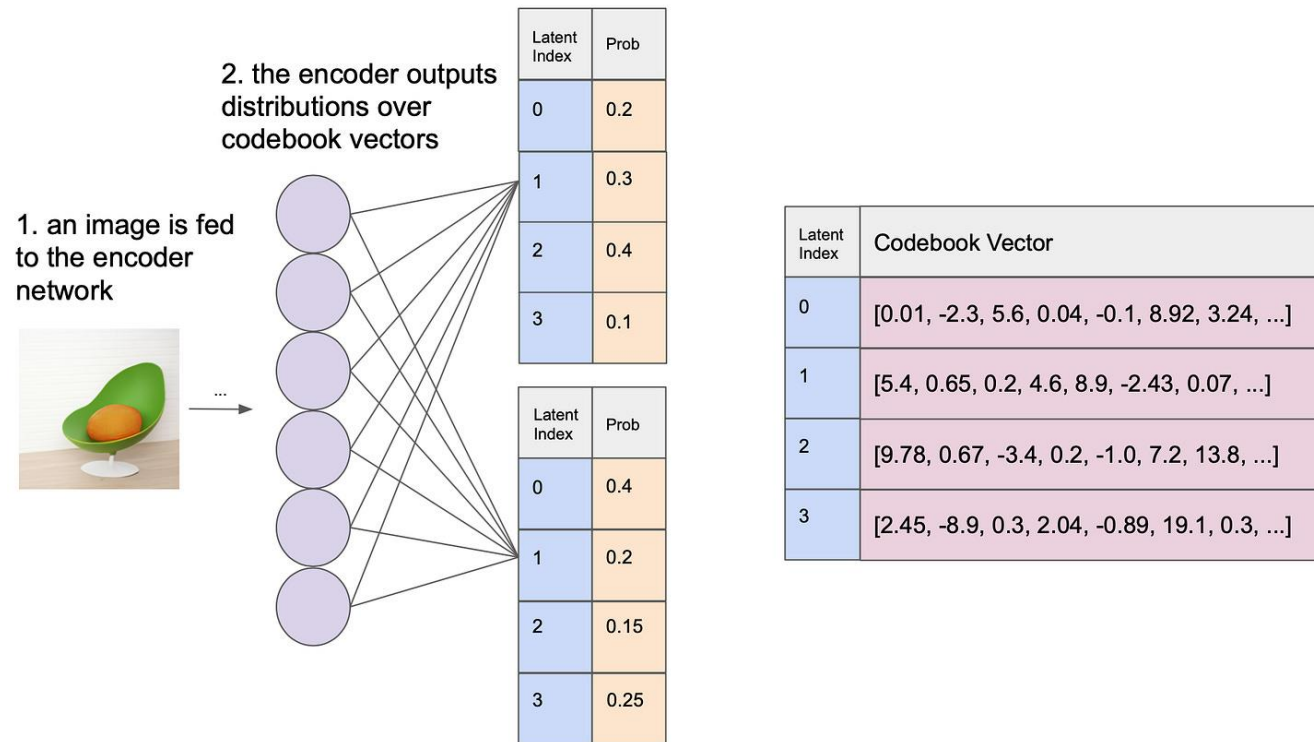


$$q(\mathbf{z} = \mathbf{e}_k | \mathbf{x}) = \begin{cases} 1 & \text{if } k = \arg \min_i \|\mathbf{z}_e(\mathbf{x}) - \mathbf{e}_i\|_2 \\ 0 & \text{otherwise.} \end{cases}$$

Pick a Codebook index by taking the closest codebook vector
The min operation require tricks to backprop

dVAE

- Instead, we can use a soft approach that predicts probability for each latent



But given the probability, how can we sample for each latent?

[DALL-E. Ramesh et al., Zero-Shot Text-to-Image Generation. ICML 2021]

[see also, Esser et al. Taming Transformers for High Resolution Image Synthesis. CVPR 2021]

The Gumbel Softmax Relaxation

- We use weighted sum

$$z = \sum_{j=1..k} y_j e_j$$

- Where

$$y_j = \text{softmax}(p), p_i \propto \frac{q(e_i|x) + g_i}{\tau}$$

- The random sampling comes by adding Gumbel distribution g_i (re-parametrization trick)

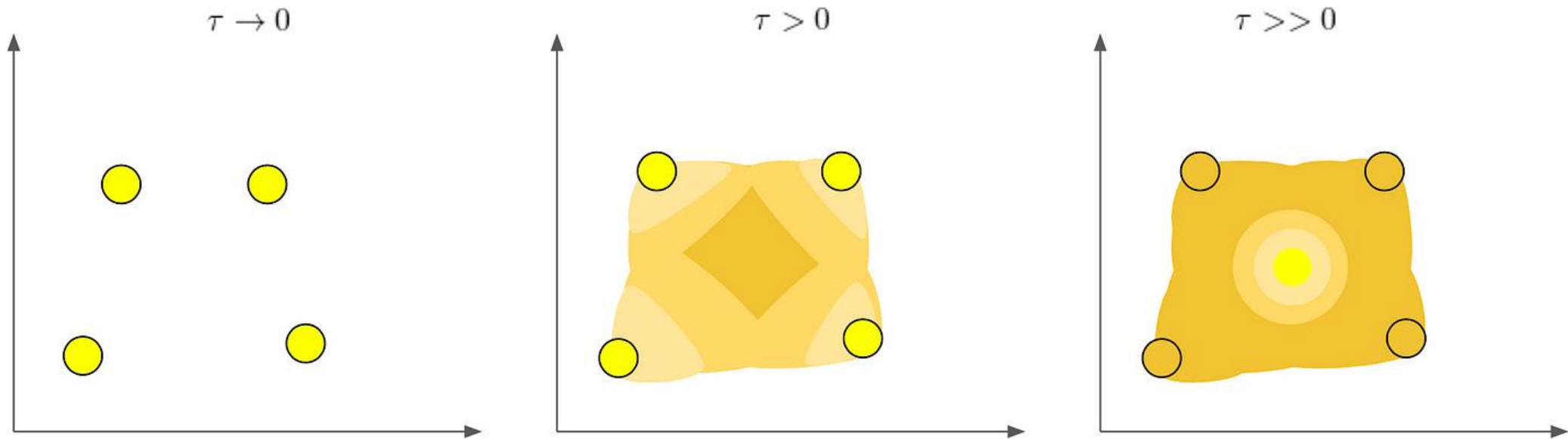
The temperature hyper-parameter

Gumbel Softmax distribution over latents for different ranges of τ

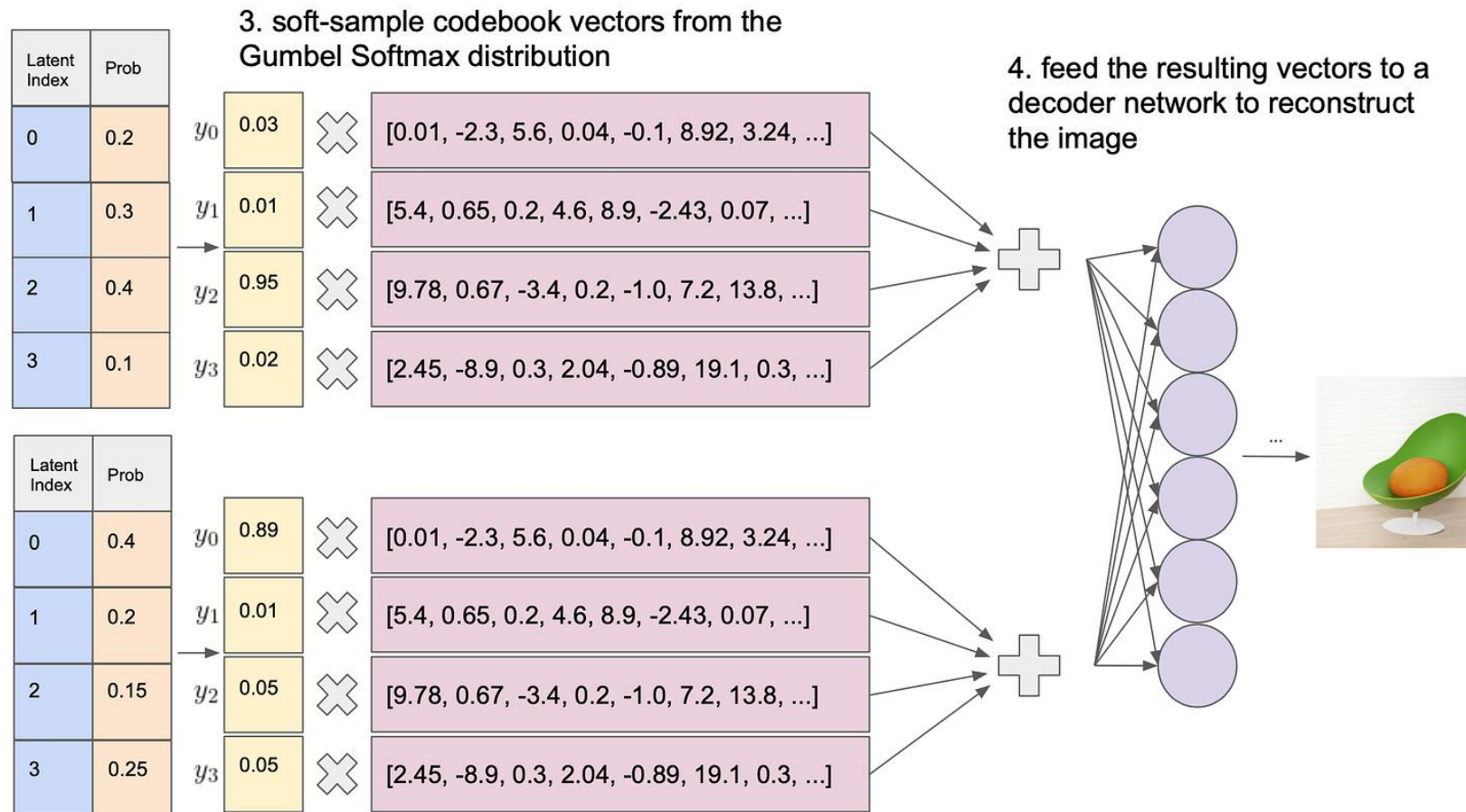
key:

 = codebook vector

brighter colors = higher probability



Encoding with dVAE



DALL-E

- DALL-E is one of the first successful
- It has 12 billion parameters, trained on 250 image-text pairs
- The main purpose is modeling $p(x|c)$; where c is a text prompt

TEXT PROMPT

an armchair in the shape of an avocado [...]

AI-GENERATED IMAGES

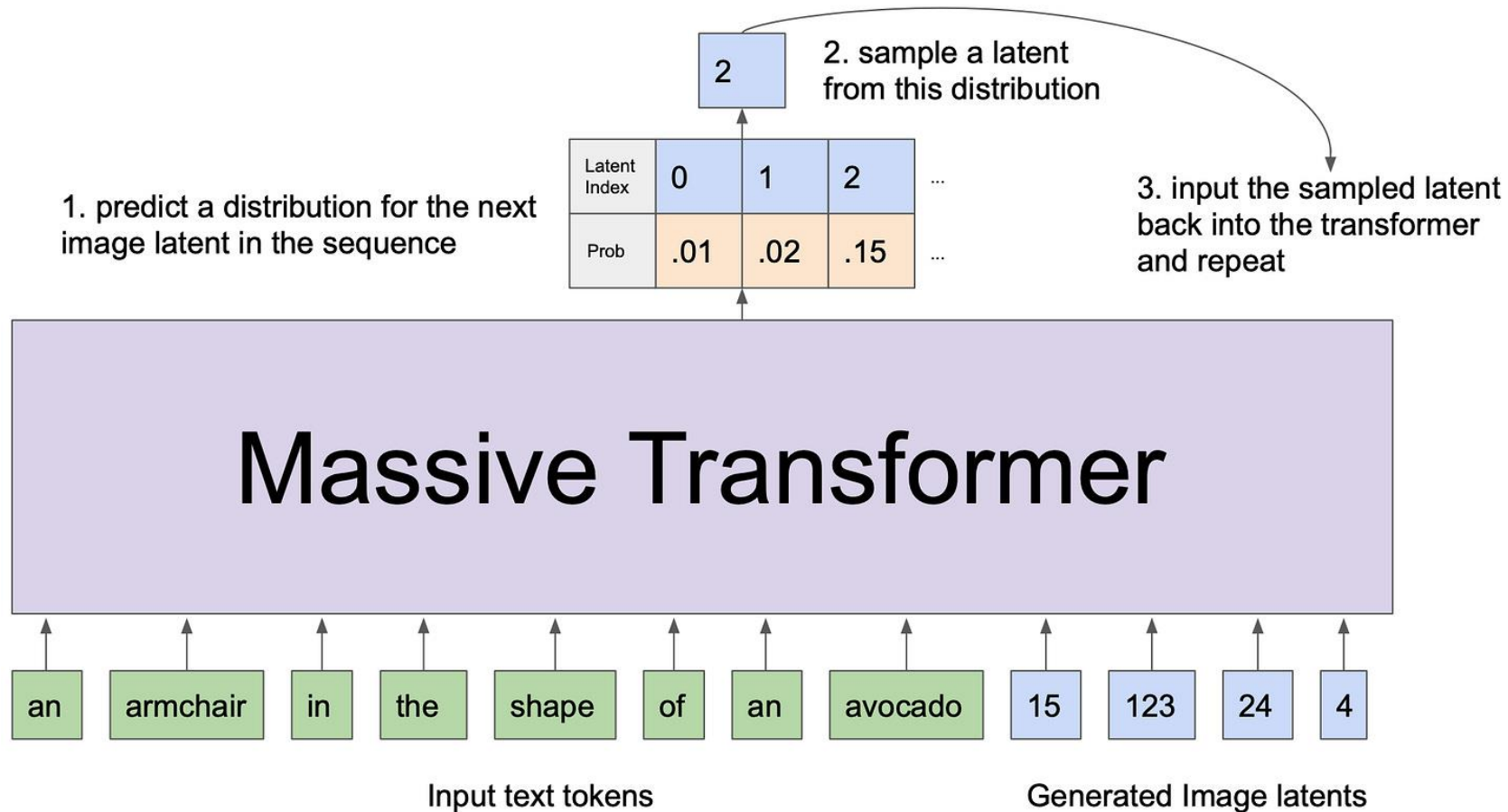


[View more or edit prompt ↓](#)

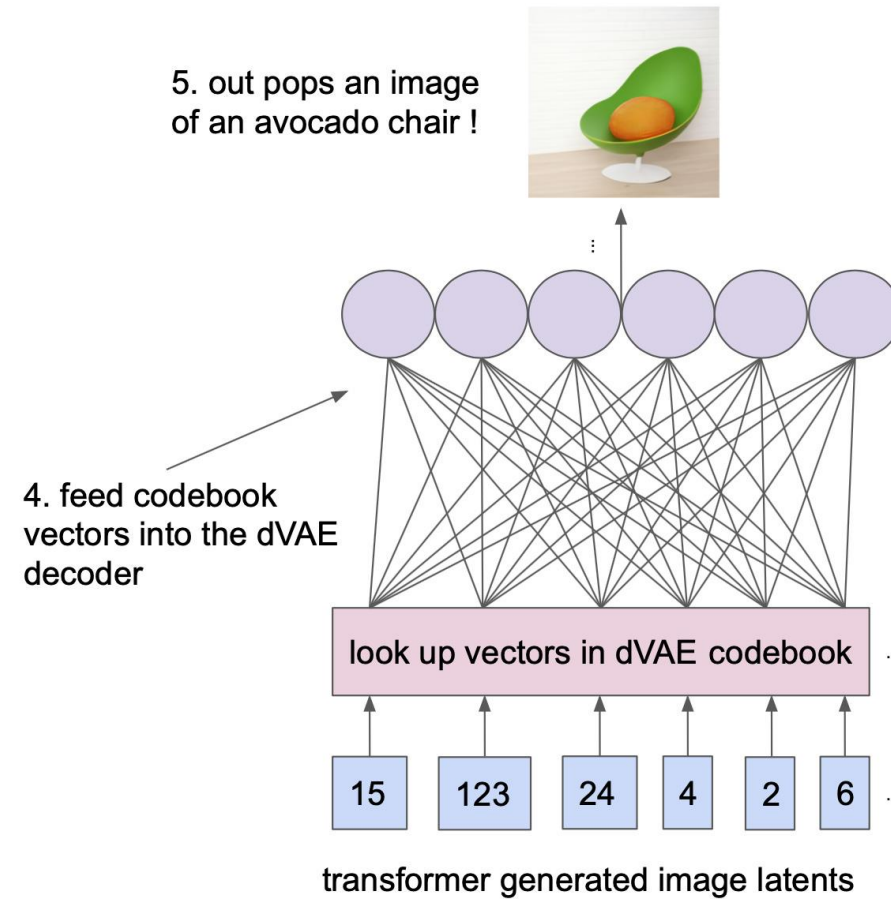
[DALL-E. Ramesh et al., Zero-Shot Text-to-Image Generation. ICML 2021]

[see also, Esser et al. Taming Transformers for High Resolution Image Synthesis. CVPR 2021]

Learn to predict codebook vectors from text



Decode with the predicted vectors



Next, diffusion process - intuition

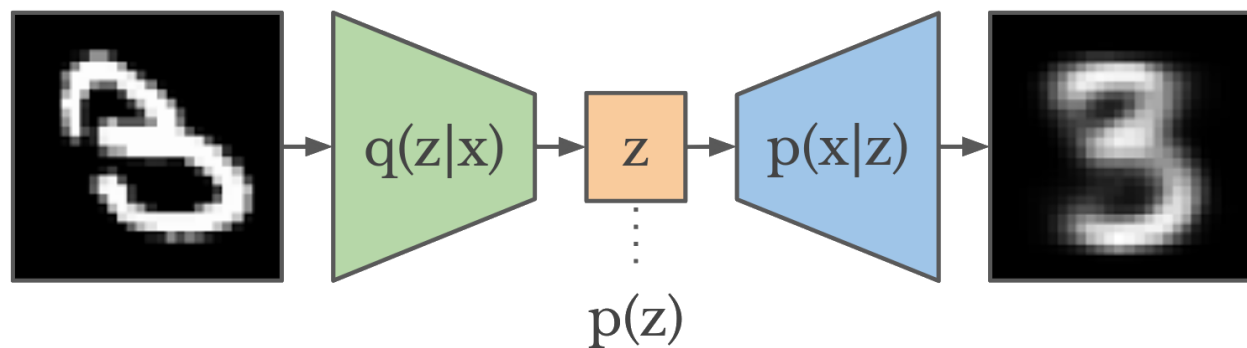
The essential idea is **to systematically and slowly destroy structure** in a data distribution through an iterative forward diffusion process.

Then **to learn a reverse diffusion process that restores structure in data**, yielding a highly flexible and tractable generative model of the data.

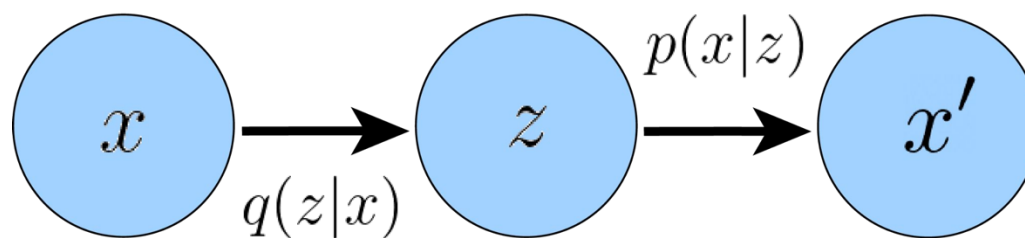
“Destroy structure” and learn the reverse process



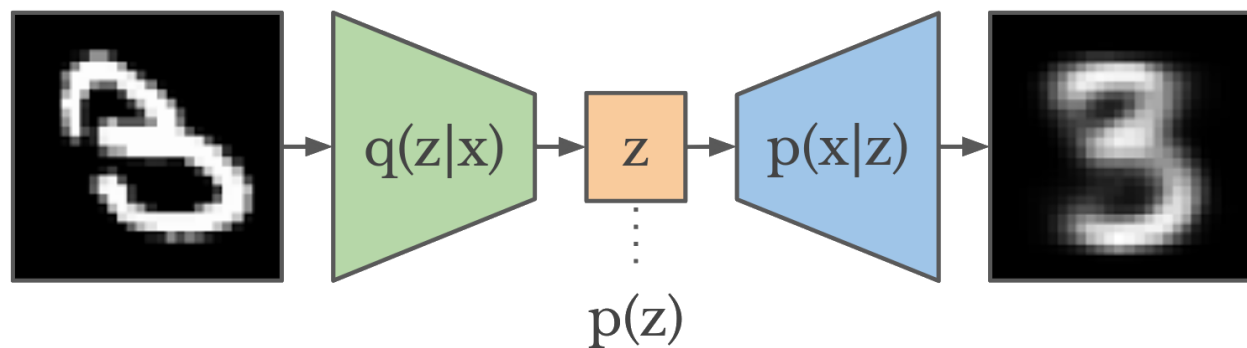
VAEs notations



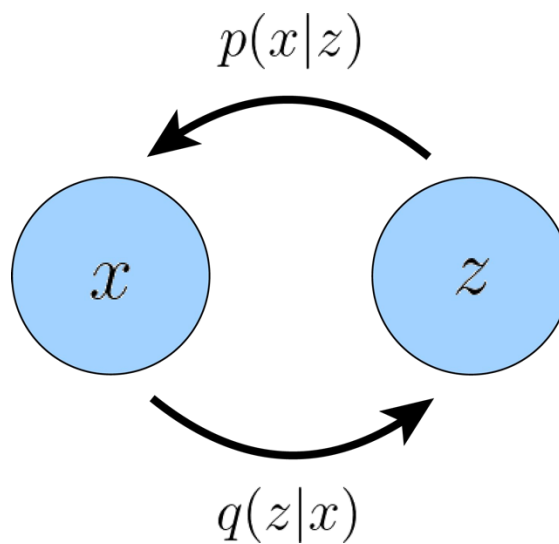
Or as



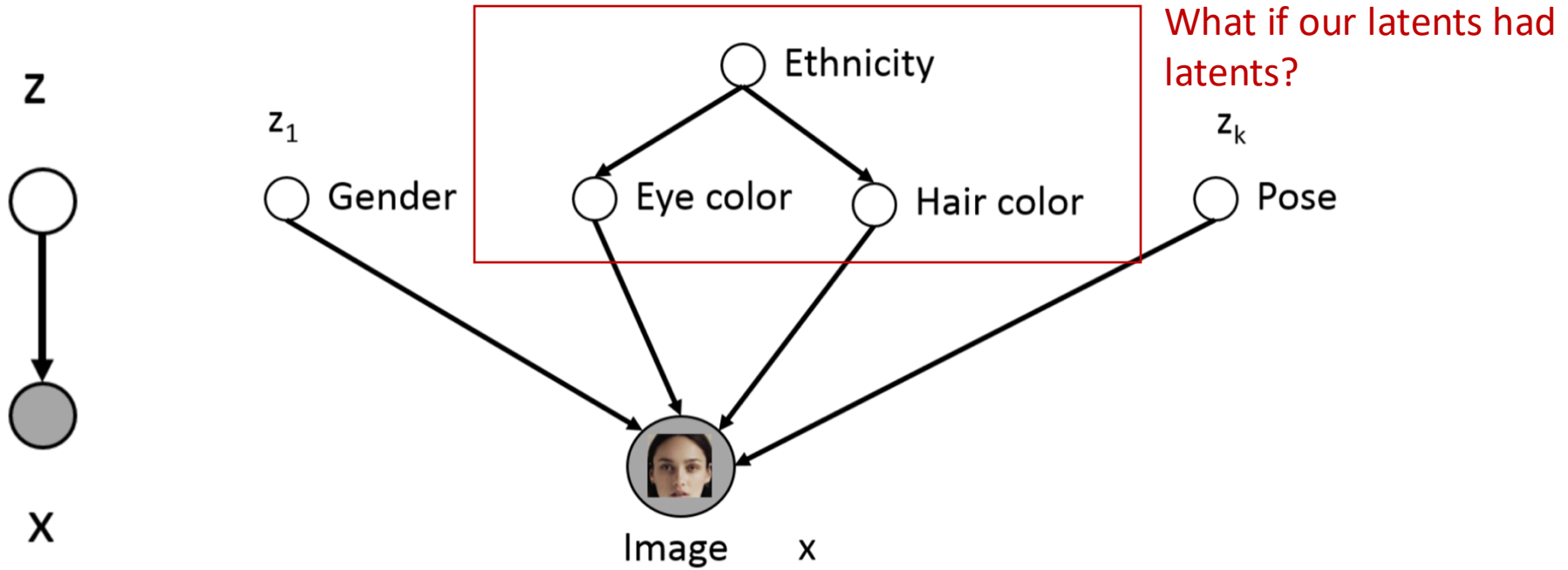
VAEs notations



Or as

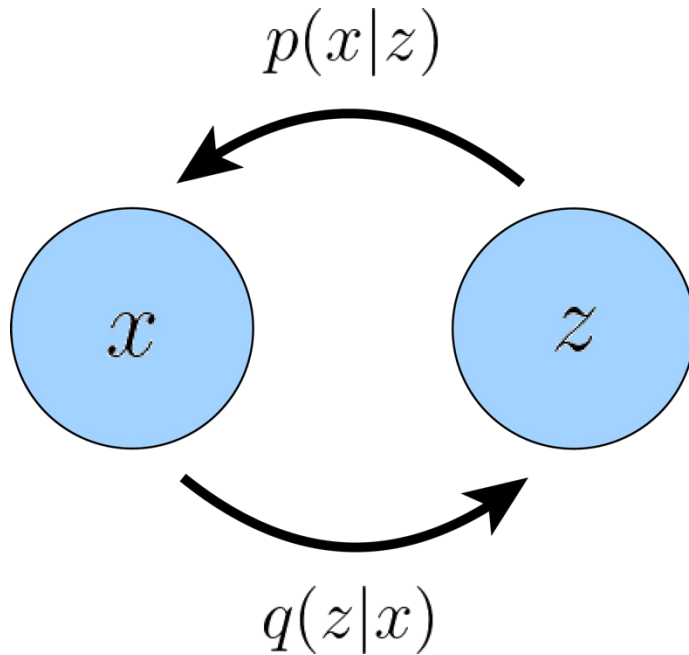


Generative Modeling with Latent Variables



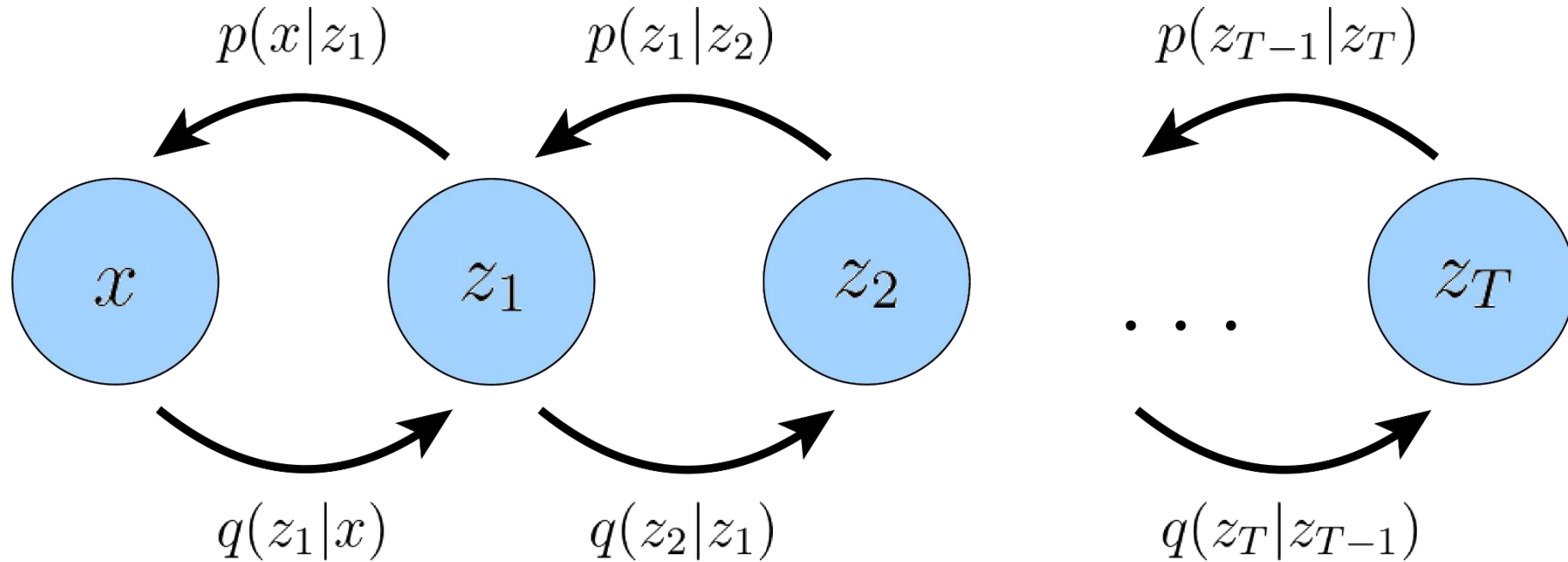
Hierarchical VAEs

- Generalize VAEs by enabling a hierarchy of latents $z = z_1, \dots, z_T$



Hierarchical VAEs

- Generalize VAEs by enabling a hierarchy of latents $z = z_1, \dots, z_T$



More on the next lecture